PCA MHEALTH · FIGURES FOR THE FINAL-YEAR REPORT

Diagrams, screens and code for the report

Use cases, data flow diagrams, activity, sequence, state and class diagrams, the database and API diagrams, the architecture, the planned and actual timelines, the screens, and the main function code, in the order of the ZCAS undergraduate project guide.

AI-Driven Mobile Health Platform for Prostate Cancer Diagnosis (Zambia)
Made on 14 October 2026 · code pictures from commit 575b547 with uncommitted changes
Drawn from the project itself. All data shown is synthetic. Not a medical device.

The diagrams, screens and code pictures for the final-year report, in the order of the chapters in the ZCAS undergraduate project guide. Each figure has a suggested number and caption, what it shows, and its files.

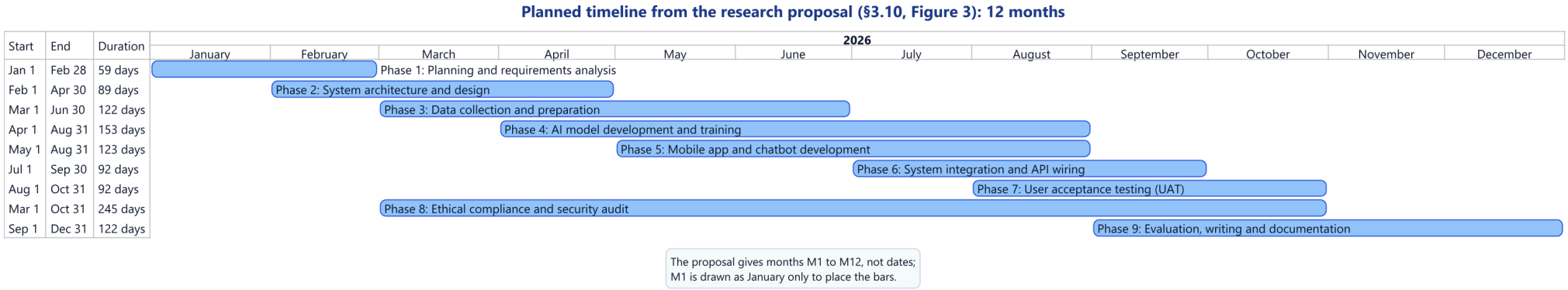
- **To put a figure in Word:** Insert › Pictures › the `.png` file. The PNGs are drawn at high resolution, so they stay sharp when printed.
- **To restyle a figure in Figma or Canva:** import the `.svg` file. It is a vector drawing, so every box and line can be moved and recoloured.
- **They are made from the project itself.** The diagrams are text files (PlantUML) in `docs/report/diagrams`. The database diagrams, the API map and the actual timeline are generated from the database schema, the access matrix and the Git history, so they cannot drift from the code. After a change, run `powershell -ExecutionPolicy Bypass -File docs\report\tools\report.ps1 diagrams figures`; report/tools/README.md explains it.
- **Honesty notes for the captions:** the AI analysis uses labelled **mock** models (their numbers come from a hash of the job id, not from patient data), all data is **synthetic**, and the knowledge base still awaits a clinician's sign-off. Say so wherever a figure shows them.
- The numbers below are suggestions; renumber them to fit your report.

Chapter 1: Introduction

1.5 Implementation plan

Figure 1.1: Planned timeline (from the research proposal)

The nine phases and twelve months of the proposal's Gantt chart (§3.10, Figure 3), redrawn. The proposal gives months M1 to M12, not dates.



Files: PNG · SVG · source gantt-planned.puml

Figure 1.2: Actual build timeline (from the Git history)

One diamond per commit, on the day it was made, from the first analysis to the latest change. Useful in Chapter 5 to compare the plan with what happened.

What was built, day by day: one diamond per commit (2026-09-23 to 2026-10-14)

Start	End	Duration	September 2026																						October 2026														
			We 23	Th 24	Fr 25	Sa 26	Su 27	Mo 28	Tu 29	We 30	Th 1	Fr 2	Sa 3	Su 4	Mo 5	Tu 6	We 7	Th 8	Fr 9	Sa 10	Su 11	Mo 12	Tu 13	We 14															
Sep 23	Sep 23	1 day	◆ Phase 0 project analysis, architecture, plan, ADRs (bd5af4b)																																			
Sep 23	Sep 23	1 day	◆ Phase 1 monorepo skeleton, quality gate and CI (6520784)																																			
Sep 23	Sep 23	1 day	◆ Phase 2 schema, migrations, audit chain, storage abstractions (f70dafe)																																			
Sep 23	Sep 23	1 day	◆ Phase 3 gateway foundation (c32a729)																																			
Sep 23	Sep 23	1 day	◆ Phase 4 authentication, sessions and RBAC (af0dbbb)																																			
Sep 24	Sep 24	1 day		◆ Phase 5 patients, clinical records and consent (d3bf0be)																																			
Sep 24	Sep 24	1 day		◆ Phase 7 Flutter foundation (d726503)																																			
Sep 24	Sep 24	1 day		◆ Phase 6 offline-first sync, Galaxy S9+ device support (5f77bd6)																																			
Sep 24	Sep 24	1 day		◆ Phase 8 patient workflow (7c3aa00)																																			
Sep 24	Sep 24	1 day		◆ Admin page, role permissions, sign-up ID, Zambian colours (c03e70d)																																			
Sep 24	Sep 24	1 day		◆ Separate React admin portal, per-user rate limits, scalability (bfd9cfc)																																			
Sep 24	Sep 24	1 day		◆ Clinical Field Health design system (5bc85f3)																																			
Sep 24	Sep 24	1 day		◆ Phase 10 imaging and histopathology files (e1959f4)																																			
Sep 25	Sep 25	1 day			◆ Phase 11 AI service foundation (f379c26)																																			
Sep 25	Sep 25	1 day			◆ Organise the repository by the six architecture layers (bfb6374)																																			
Sep 25	Sep 25	1 day			◆ Phase 12 explainability pipeline and evaluation endpoint (0fd06c1)																																			
Sep 25	Sep 25	1 day			◆ AI model integration guide and chatbot build plan (beb163b)																																			
Sep 25	Sep 25	1 day			◆ Phase 9 clinician and pathologist workflow (e85f15a)																																			
Sep 28	Sep 28	1 day						◆ Dark mode and settings screen (0aebf4e)																																	
Sep 28	Sep 28	1 day						◆ Phase 14 de-identified FHIR R4 export and SmartCare Pro mock (dae7661)																																	
Sep 28	Sep 28	1 day						◆ Phase 15 security and compliance review (3bbb4ec)																																	
Sep 28	Sep 28	1 day						◆ Activity dashboard for the phone app and a new navigation bar (967c988)																																	
Sep 28	Sep 28	1 day						◆ "modern health app" redesign, all screens, light and dark (12c1d57)																																	
Sep 28	Sep 28	1 day						◆ Phase 16 end-to-end workflows on the live system (763f1a8)																																	
Sep 29	Sep 29	1 day							◆ Phase 17 performance measured against the research targets (a70ecc8)																																	
Sep 29	Sep 29	1 day							◆ Phase 13 milestone 1, knowledge base and offline extractive answers (21b9634)																																	
Sep 29	Sep 29	1 day							◆ Phase 13 milestone 2, chat API with safety rules (2841f7b)																																	
Sep 29	Sep 29	1 day							◆ Phase 13 milestone 3, "Ask a question" and "Ask the assistant" in the app (5f3624f)																																	
Sep 29	Sep 29	1 day							◆ Phase 13 measured and documented (26dcb9b)																																	
Sep 29	Sep 29	1 day							◆ The assistant chats, and Claude can write its answers (7b91756)																																	
Sep 29	Sep 29	1 day							◆ Plain words for chat actions in the dashboard feed (6150f22)																																	
Sep 29	Sep 29	1 day							◆ Awareness blue, modern icons, assistant bot, voice messages and read-aloud (1b32380)																																	
Sep 29	Sep 29	1 day							◆ Use the PC's own PostgreSQL 18 and MongoDB 8.3 (395441a)																																	
Sep 29	Sep 29	1 day							◆ Delete accounts from the admin website (47dcffc)																																	
Sep 29	Sep 29	1 day							◆ New chats, past chats and casual conversation (3fae63c)																																	
Sep 30	Sep 30	1 day								◆ Operations manual: how it was built and how to run it without an assistant (485825f)																																
Sep 30	Sep 30	1 day								◆ Dev-up.ps1: start when Docker Desktop is not running (bba633c)																																
Sep 30	Sep 30	1 day								◆ Development log: chat features checked on the emulator; voice recognition s... (9cdccac)																																
Oct 1	Oct 1	1 day									◆ Operations manual: a tool to rebuild its pictures, PDF and web page (dd1813e)																																
Oct 1	Oct 1	1 day									◆ Diagrams and figures for the final-year report; proposal gaps listed (0fde88c)																																
Oct 1	Oct 1	1 day									◆ User acceptance testing kit (proposal §3.8.2, NFR-04) (9104c32)																																
Oct 1	Oct 1	1 day									◆ Meaning search with MedCPT next to the keyword search (ADR-013) (b3dd3d4)																																
Oct 2	Oct 2	1 day										◆ Push notifications through Firebase Cloud Messaging (ADR-014) (41fd514)																																
Oct 2	Oct 2	1 day										◆ Meaning chooses the article, keywords the section (ADR-013 fix) (3c1c619)																																
Oct 2	Oct 2	1 day										◆ Check out text files with LF line endings on every platform (b304748)																																
Oct 2	Oct 2	1 day										◆ Generate the Prisma client on install; validate without a database (8af29d1)																																
Oct 2	Oct 2	1 day										◆ A lockout no longer ends sessions that are already signed in (a815f2f)																																
Oct 2	Oct 2	1 day										◆ A malformed refresh cookie gets 401, not a server error (dc0fb06)																																
Oct 2	Oct 2	1 day										◆ DB test: a lockout leaves open sessions working (4e22352)																																
Oct 2	Oct 2	1 day										◆ Review of 2 October 2026: checks run, defects fixed, findings (80e73e1)																																
Oct 3	Oct 3	1 day											◆ Review follow-up: pushes stop when sessions end; visible field outlines (d4467b4)																															
Oct 3	Oct 3	1 day											◆ Linking an app account has the admin page's safeguards (M-2) (e106c34)																															
Oct 14	Oct 14	1 day																							◆ Claude chat answers work with anthropic 1.x (ADR-010) (575b547)															

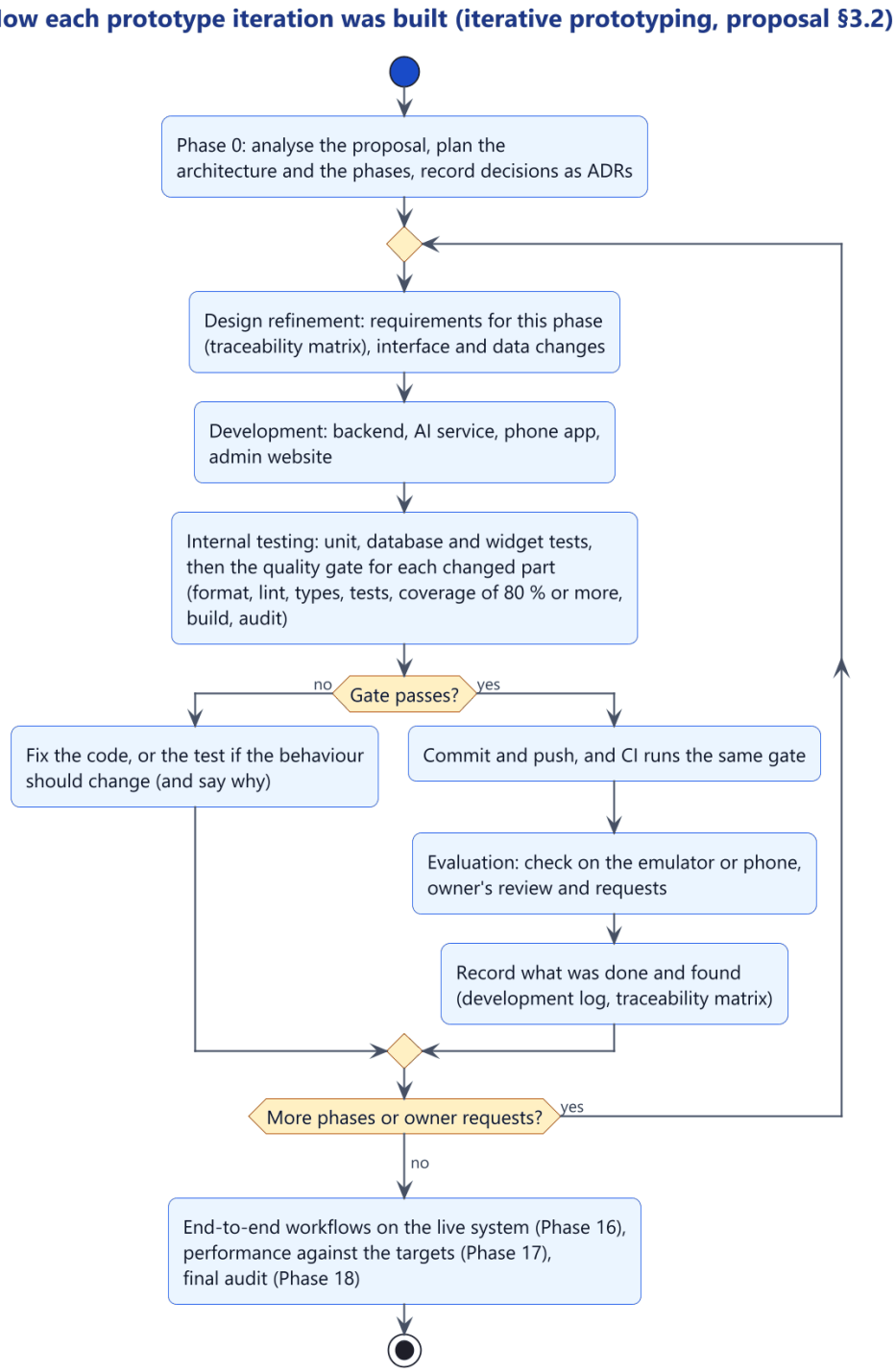
Files: PNG · SVG · source generated from `git log`

Chapter 3: System analysis and design

3.3 System development method

Figure 3.1: The iterative prototyping cycle as followed

How each phase went through design refinement, development, internal testing (the quality gate) and evaluation, as in proposal §3.2.

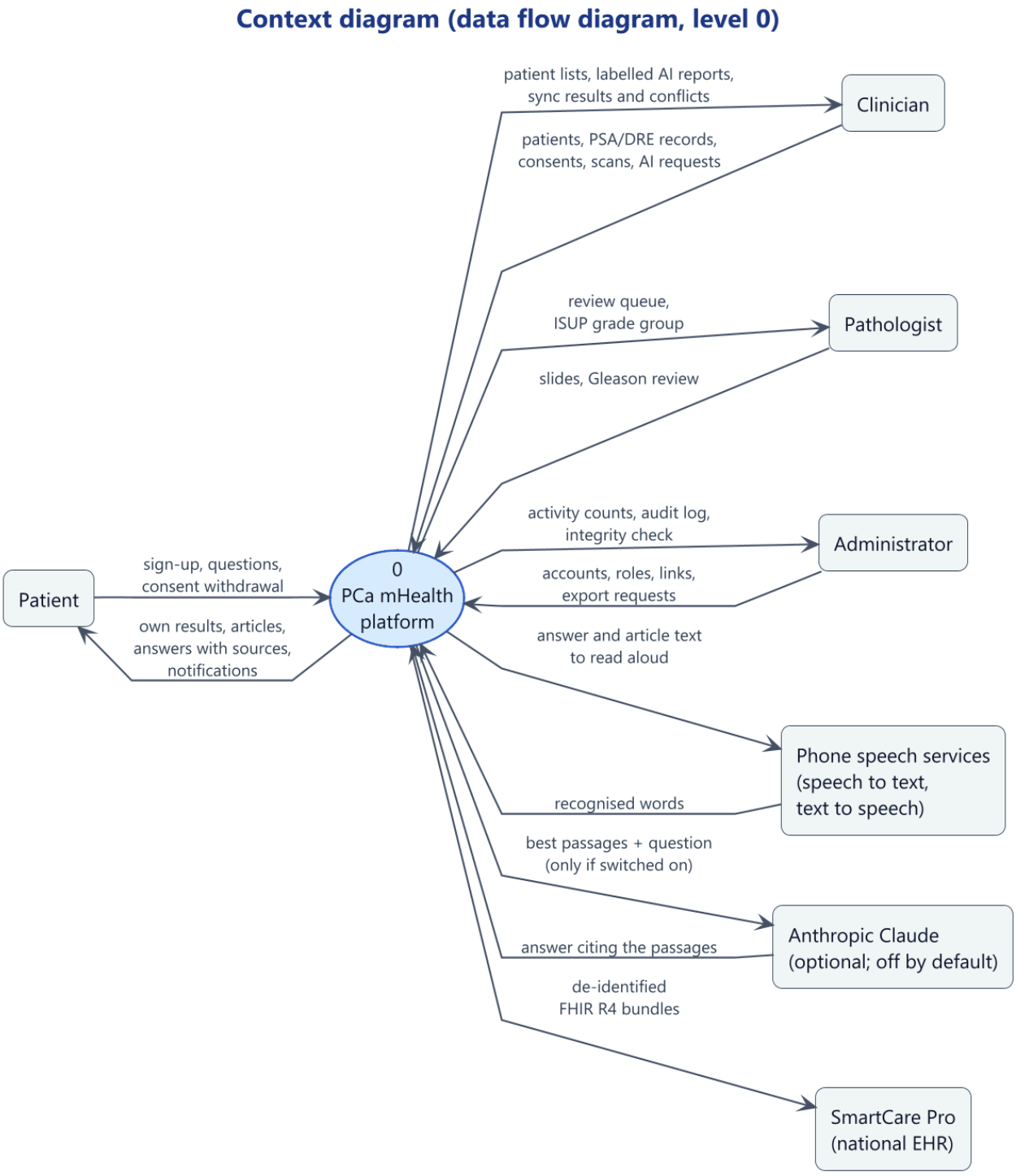


Files: PNG · SVG · source activity-prototyping-cycle.puml

3.4 Data and process modelling

Figure 3.2: Context diagram (DFD level 0)

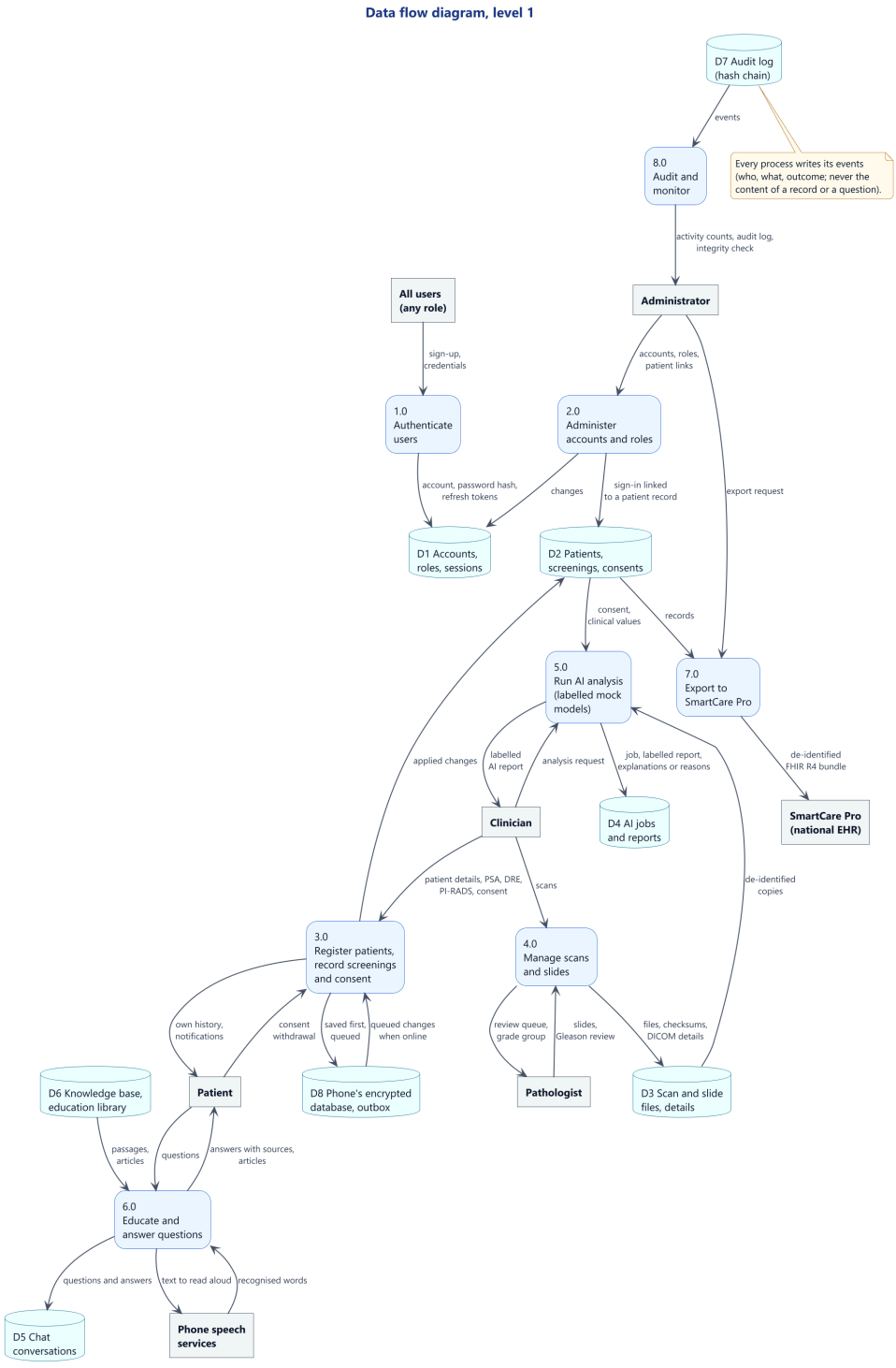
The whole platform as one process, with the people and systems around it and the data that flows between them.



Files: PNG · SVG · source context-diagram.puml

Figure 3.3: Data flow diagram, level 1

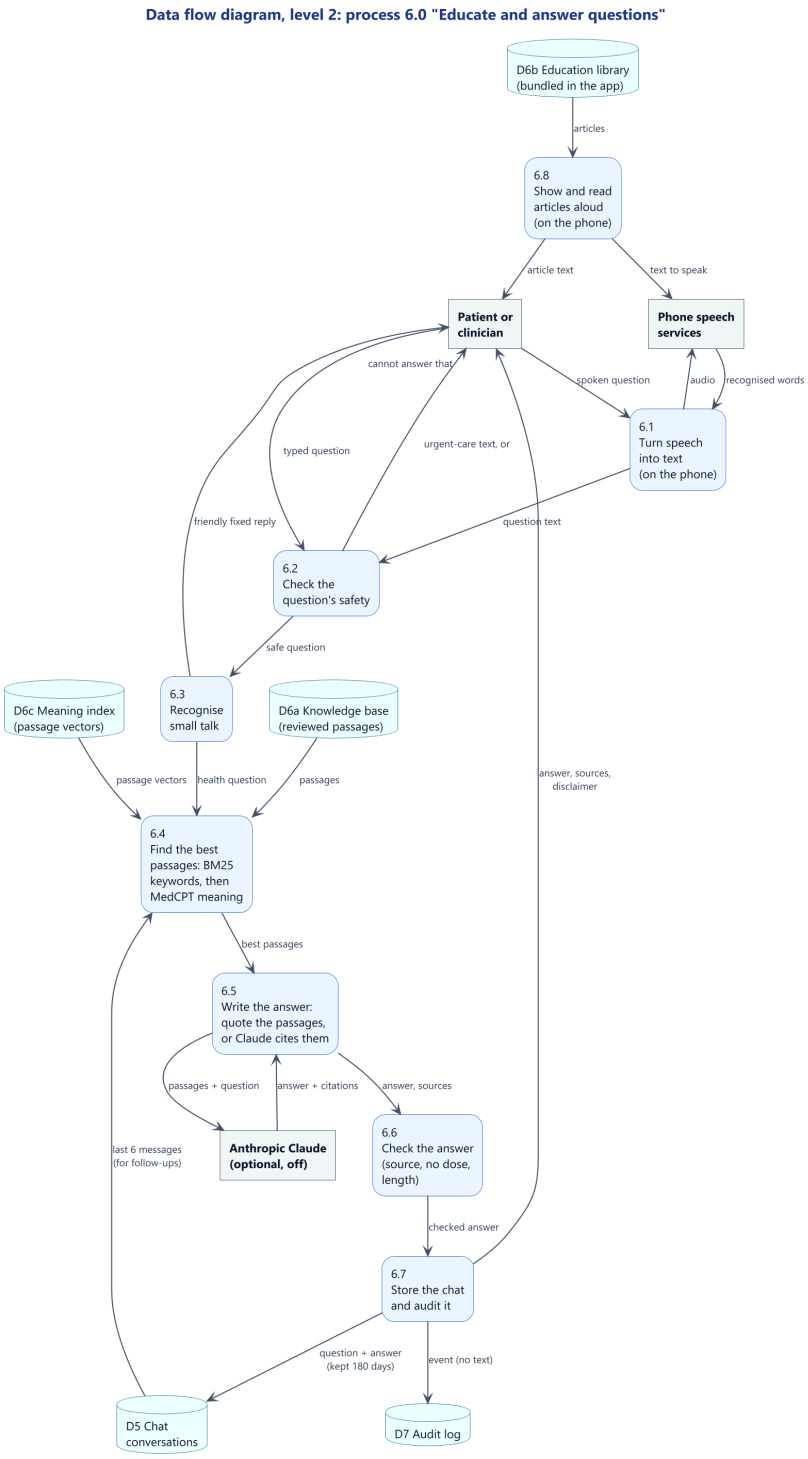
The eight main processes, the eight data stores (D1 to D8) and the flows between them and the users.



Files: PNG · SVG · source dfd-level-1.puml

Figure 3.4: Data flow diagram, level 2: the assistant and education

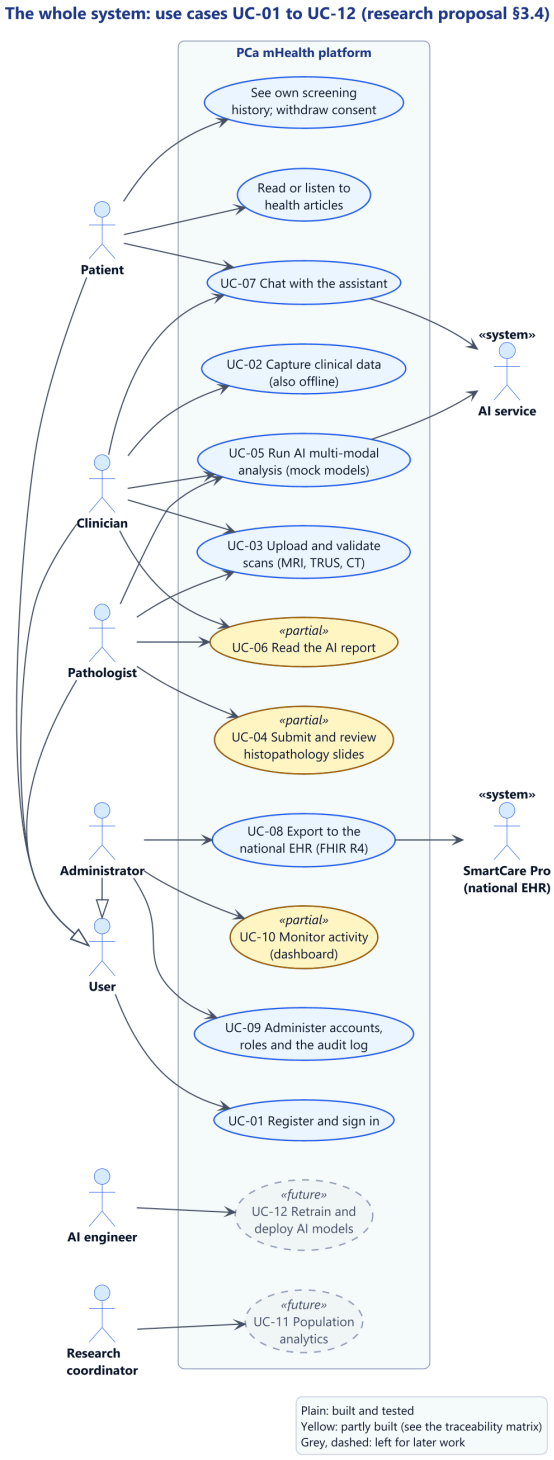
Process 6.0 in detail: speech to text on the phone, the safety check, small talk, the knowledge-base search, writing and checking the answer, storing it, and reading articles aloud.



Files: PNG · SVG · source dfd-level-2-chatbot.puml

Figure 3.5: Use case diagram: the whole system (UC-01 to UC-12)

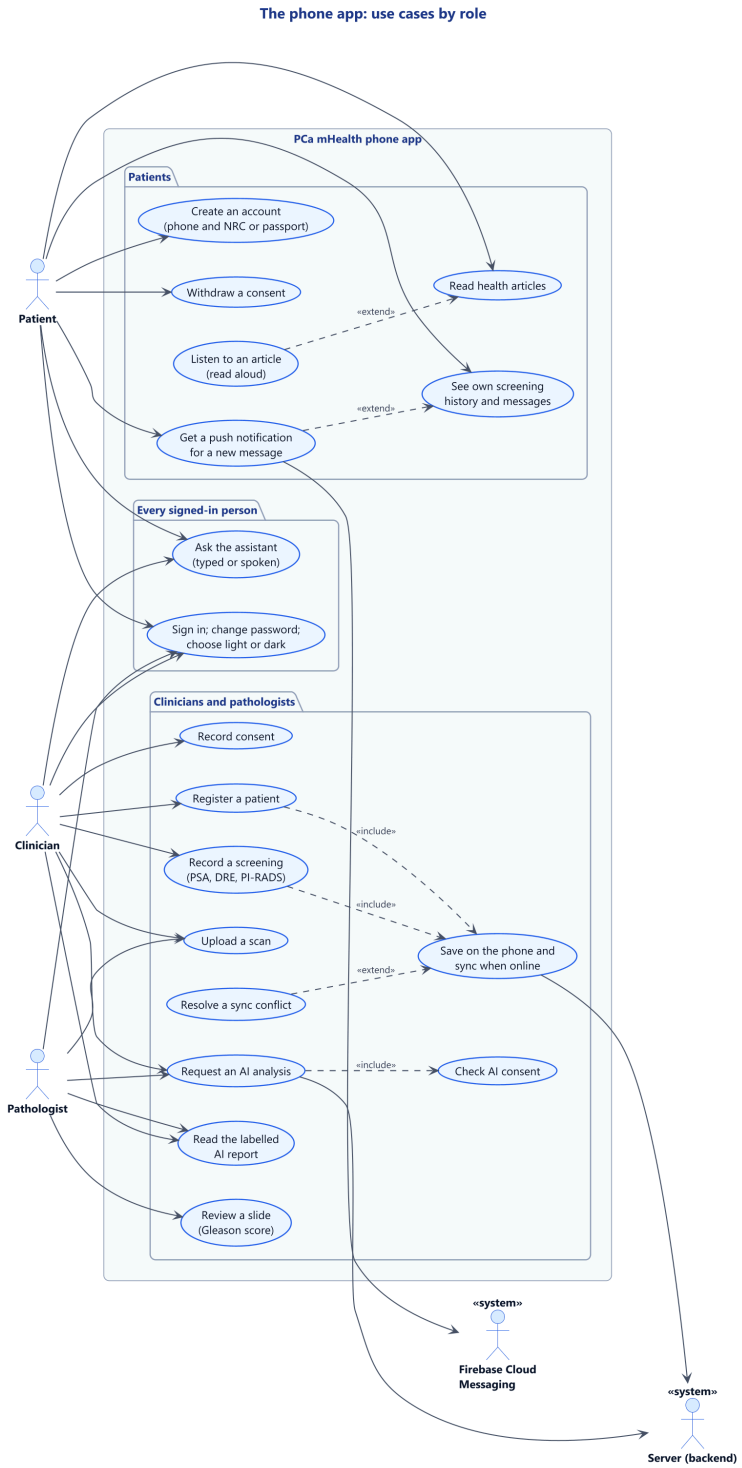
The twelve use cases of the proposal (§3.4) with their actors. Yellow: partly built; grey and dashed: left for later work.



Files: PNG · SVG · source usecase-system.puml

Figure 3.6: Use case diagram: the phone app

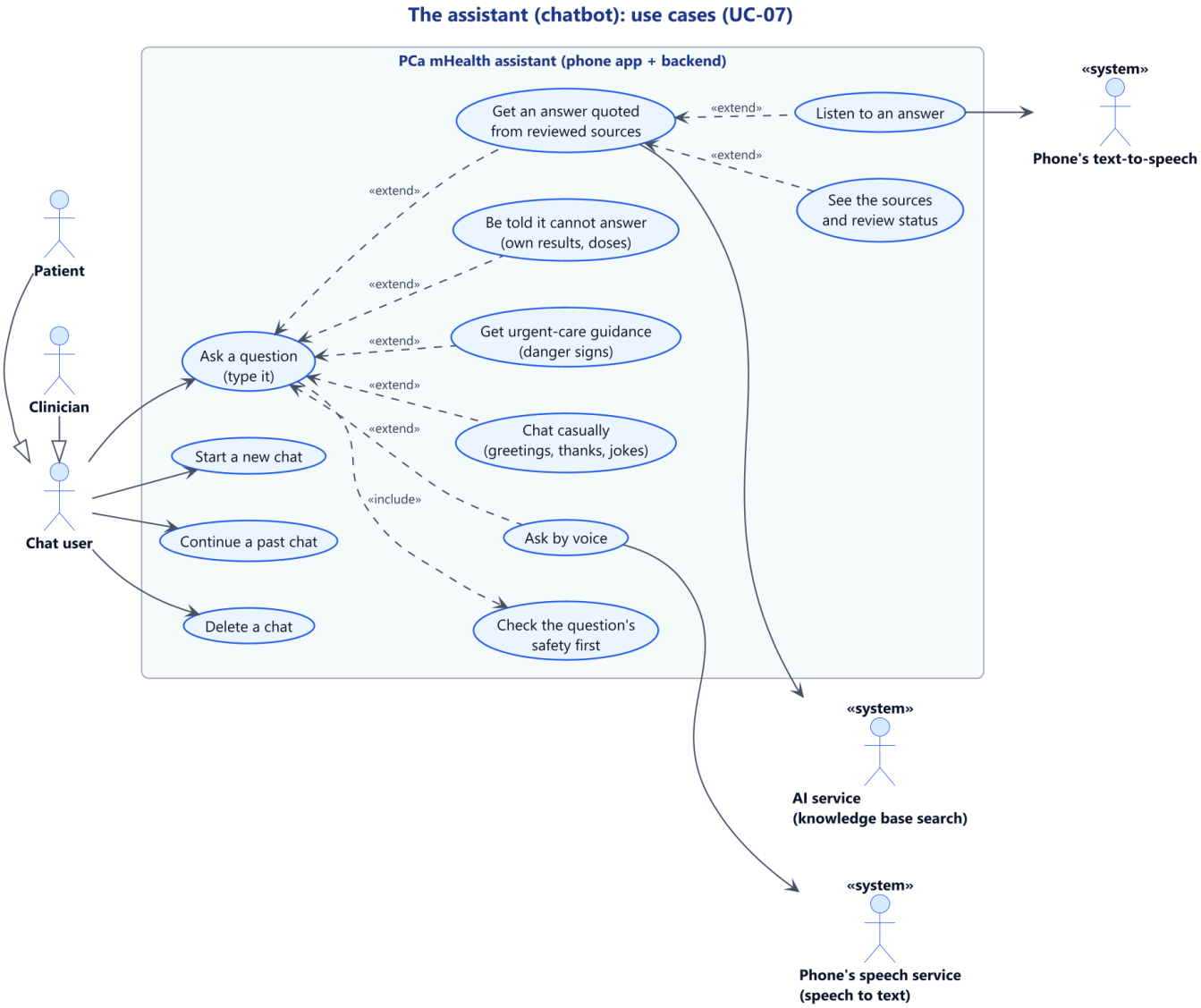
What patients, clinicians and pathologists can do in the app, with the include and extend relations (for example, saving on the phone and syncing is included in registering a patient).



Files: PNG · SVG · source usecase-mobile-app.puml

Figure 3.7: Use case diagram: the assistant (UC-07)

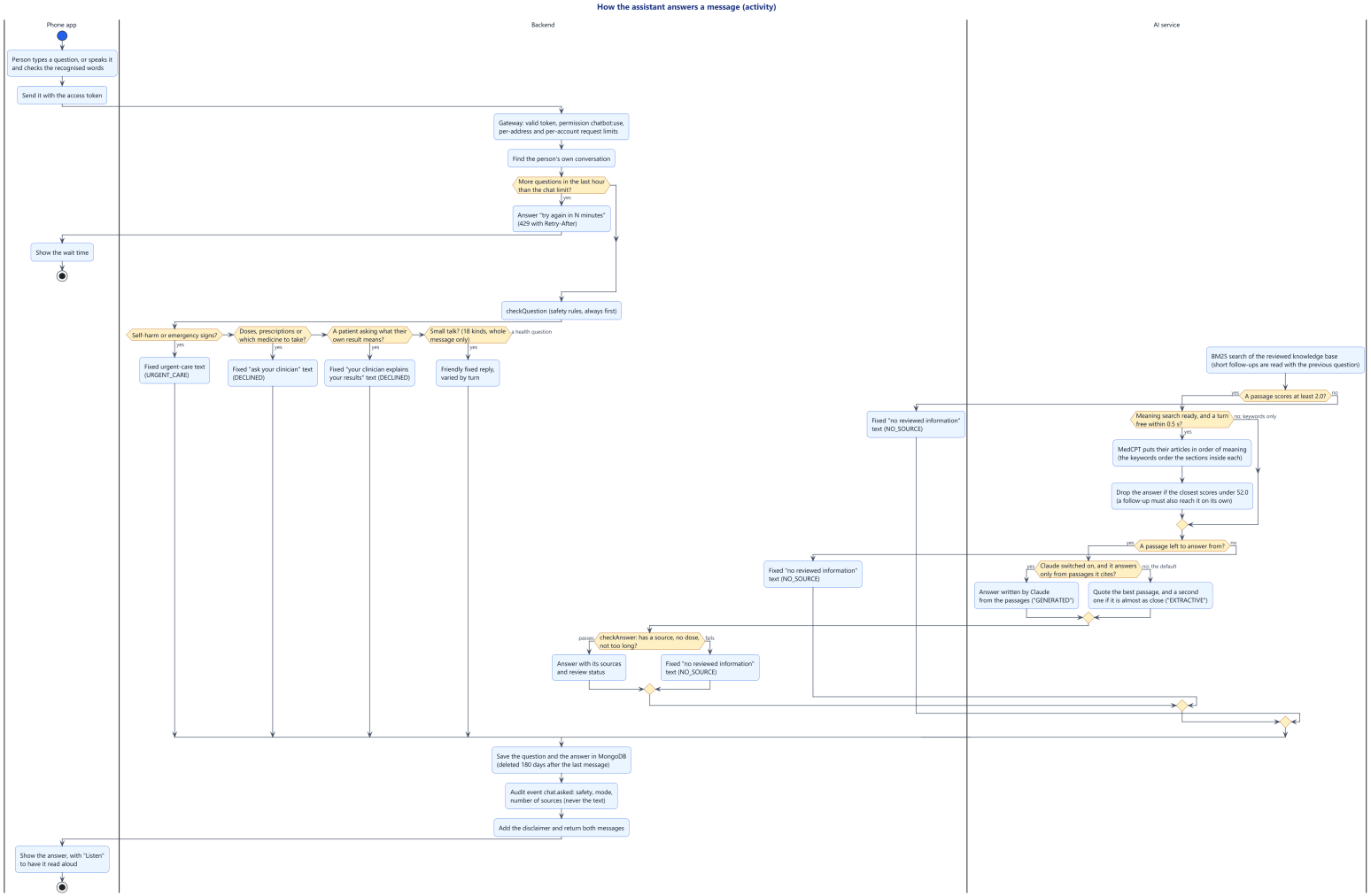
Asking by typing or voice, the safety check that is always included, the kinds of answers, listening to an answer, and managing chats.



Files: PNG · SVG · source usecase-chatbot.puml

Figure 3.8: Activity diagram: how the assistant answers a message

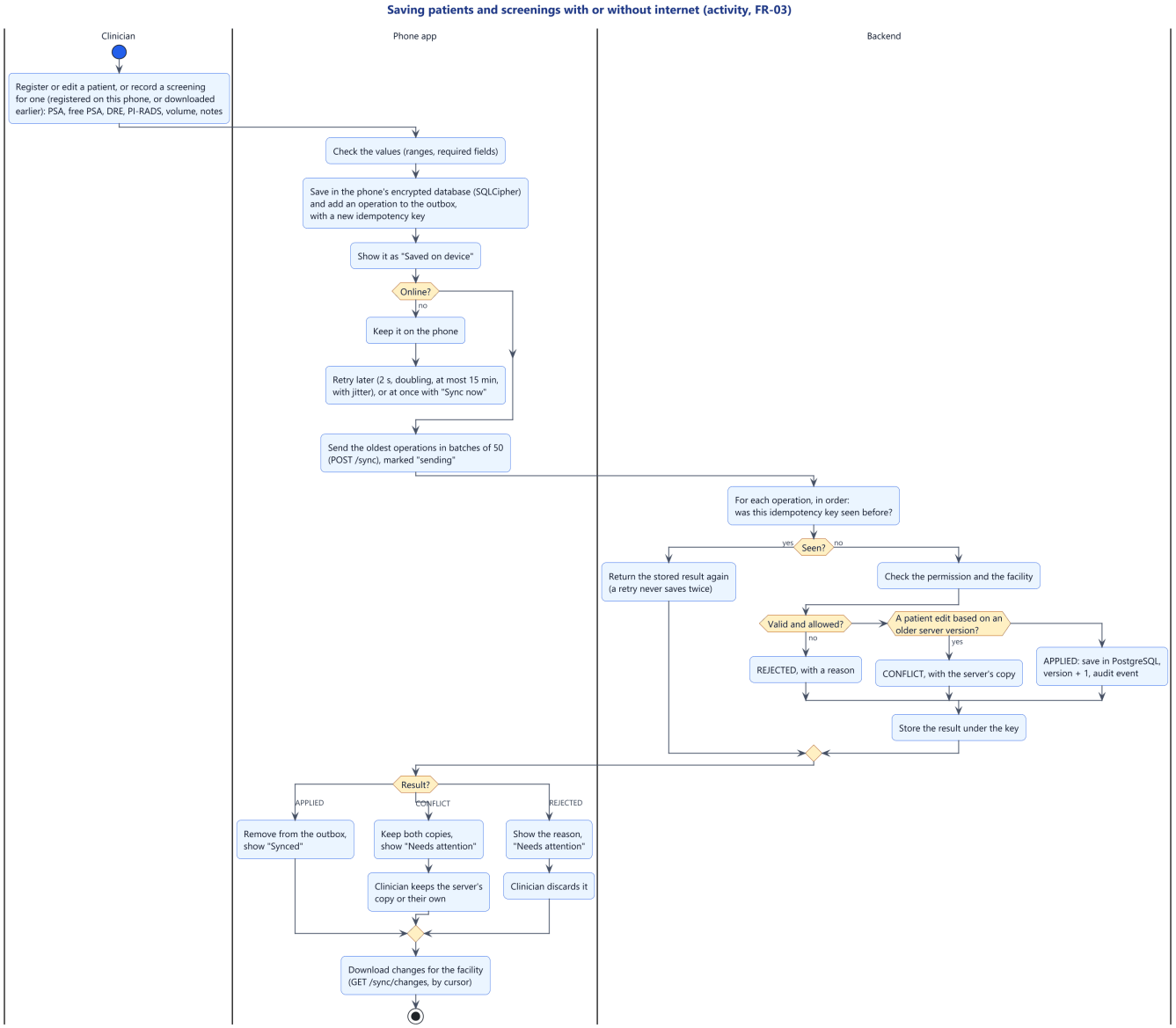
Every decision from the request limit to the stored answer: urgent-care signs, medicines, a patient's own results, small talk, the knowledge-base search (keywords, then meaning) and the answer check.



Files: PNG · SVG · source activity-chat-answer.puml

Figure 3.9: Activity diagram: saving patients and screenings with or without internet

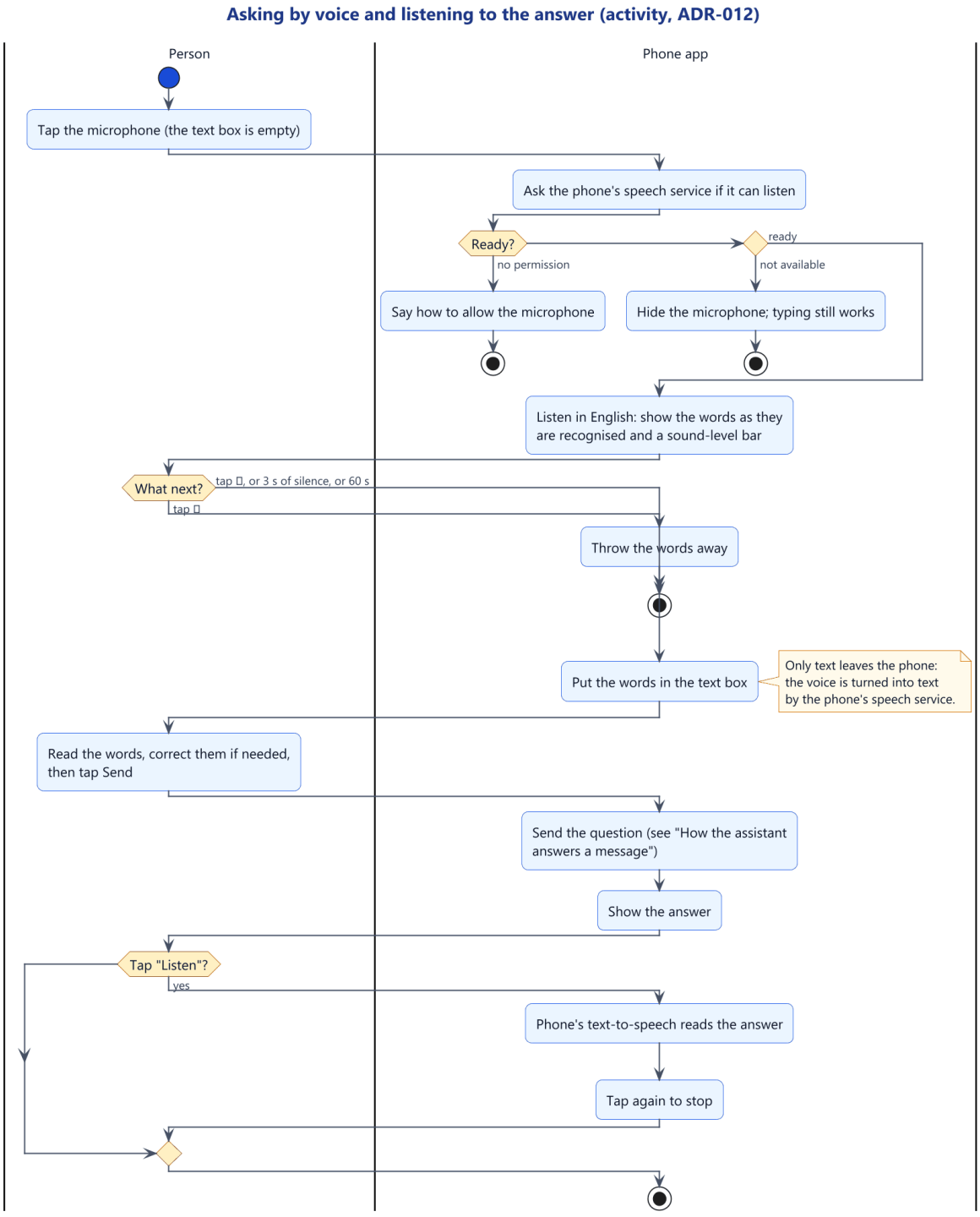
The offline-first design (FR-03): saved on the phone first, sent in batches with idempotency keys, and the three results (applied, conflict, rejected).



Files: PNG · SVG · source activity-offline-capture.puml

Figure 3.10: Activity diagram: asking by voice and listening to the answer

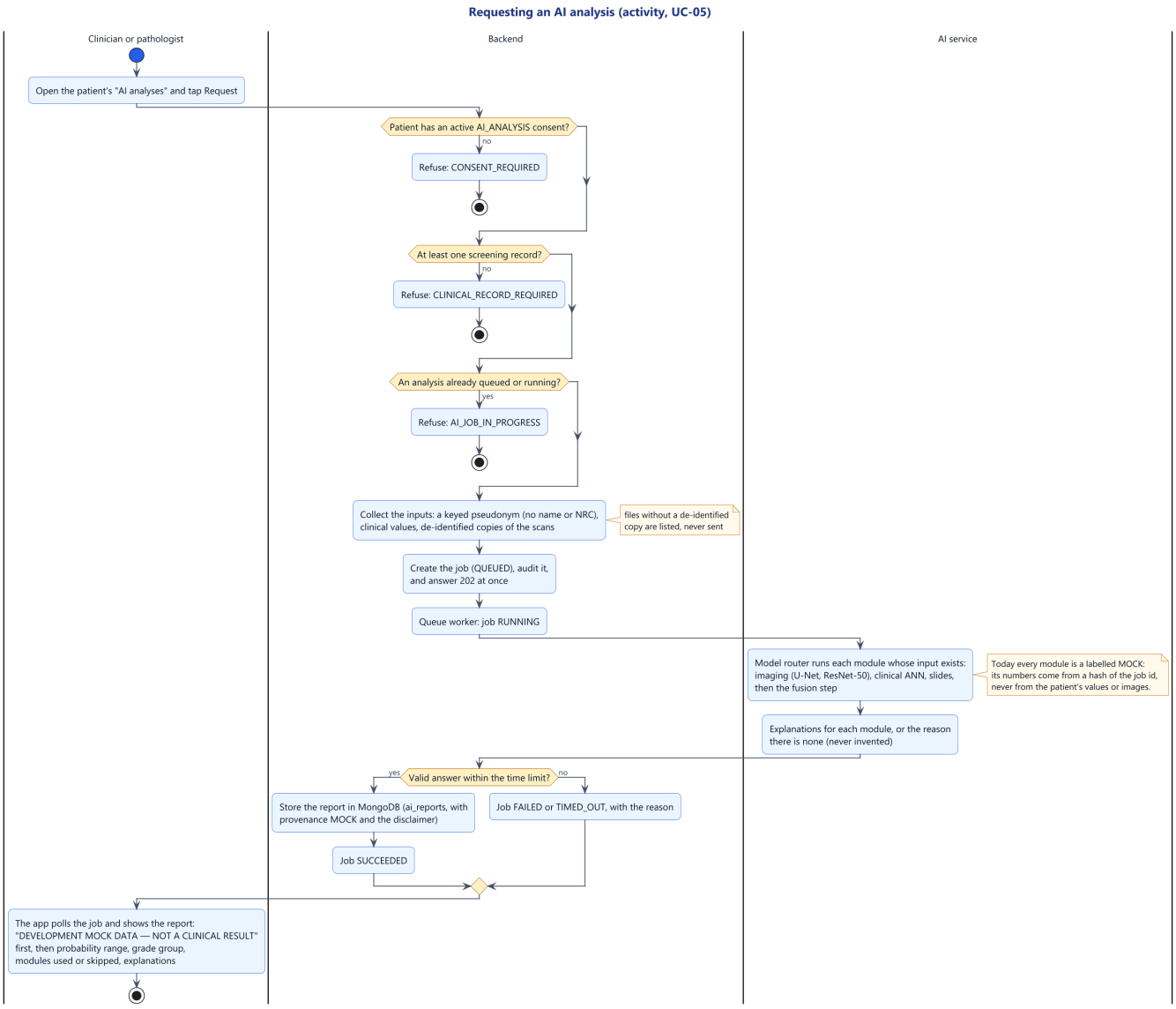
Voice input with the phone's own speech service, and read-aloud (ADR-012).



Files: PNG · SVG · source activity-voice-question.puml

Figure 3.11: Activity diagram: requesting an AI analysis

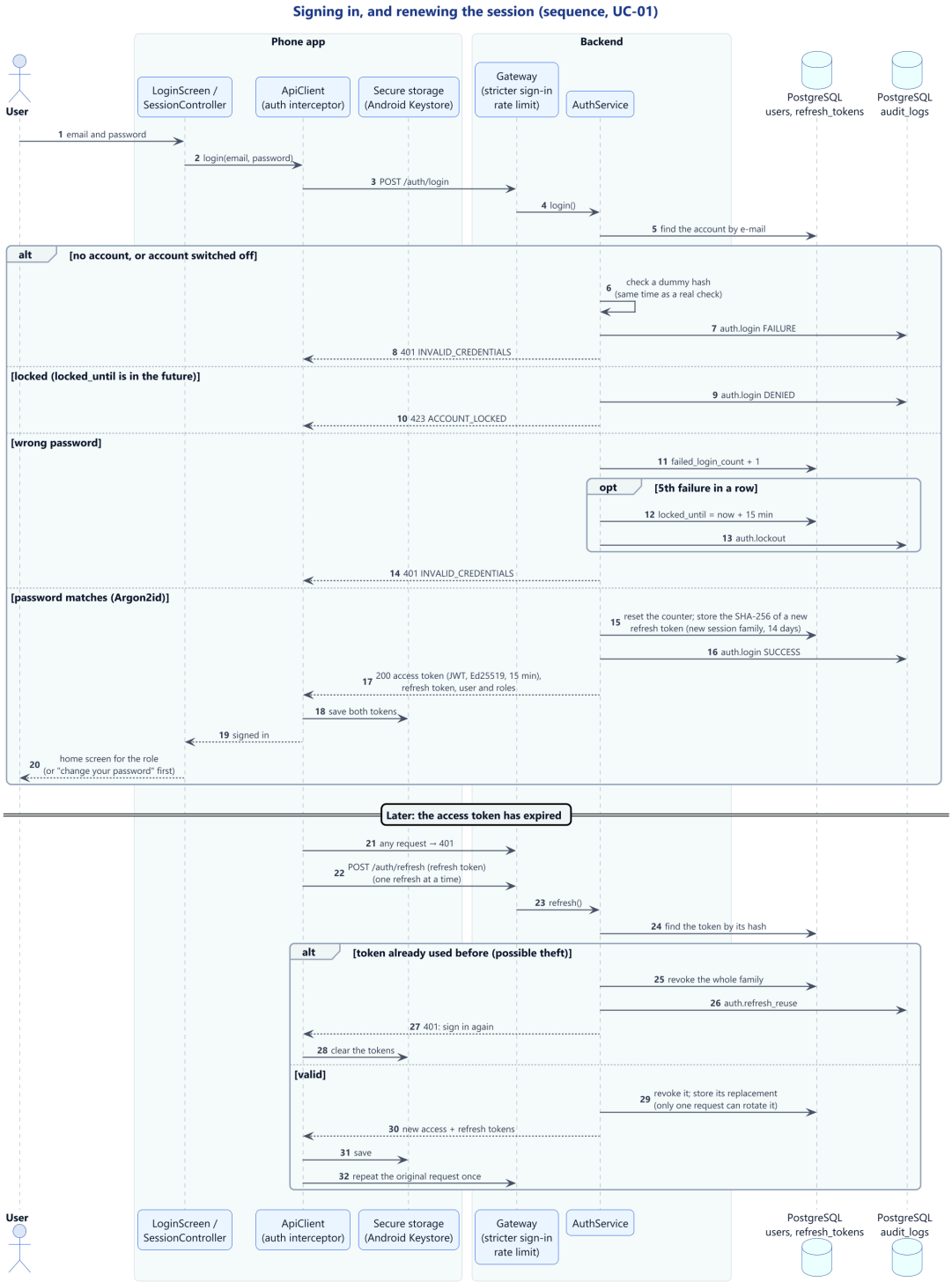
The consent, record and one-at-a-time checks, the de-identified inputs, the queue, and the labelled mock result.



Files: PNG · SVG · source activity-ai-analysis.puml

Figure 3.12: Sequence diagram: signing in and renewing the session

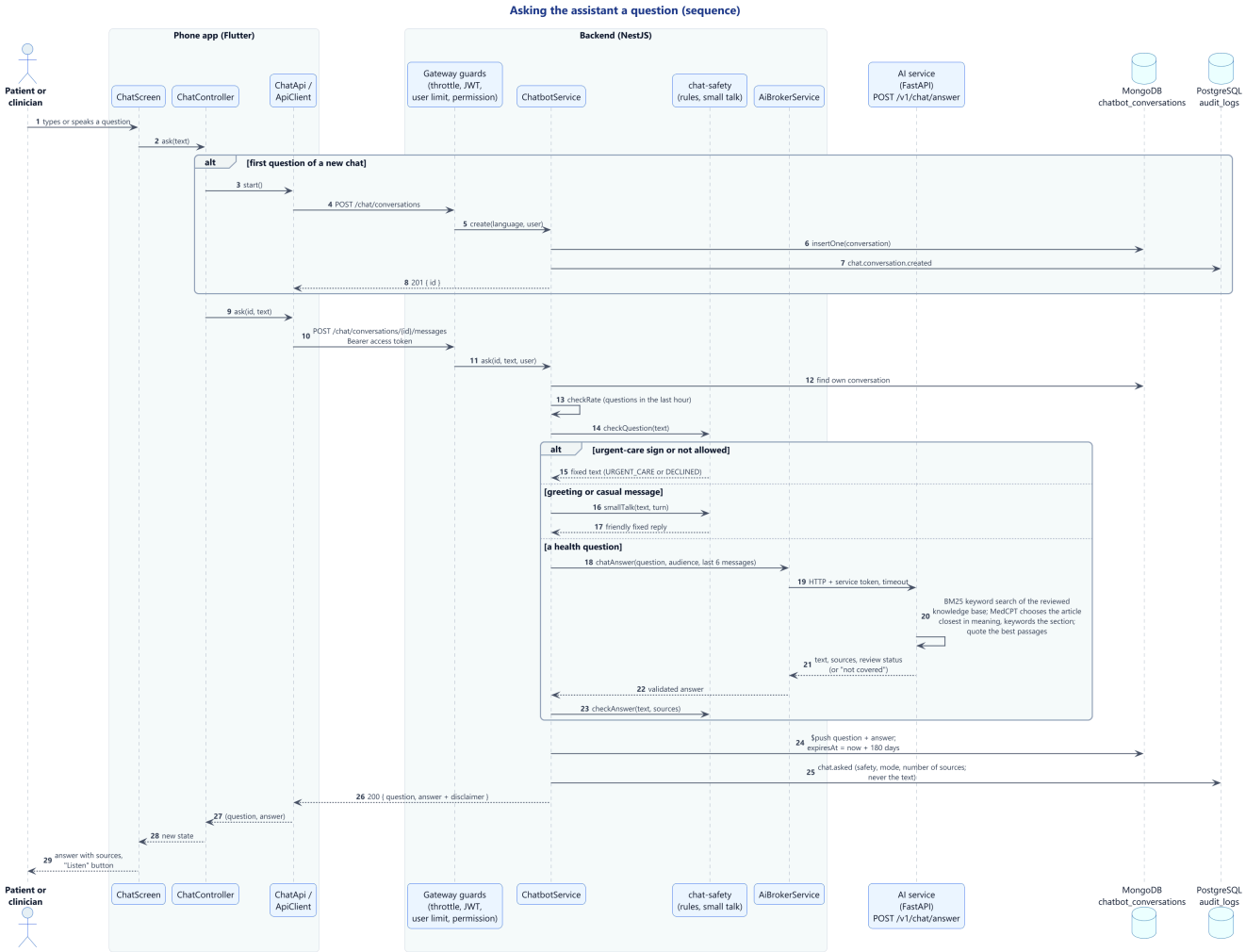
Sign-in with lockout after five failures, the short-lived access token, and refresh-token rotation with theft detection.



Files: PNG · SVG · source sequence-sign-in.puml

Figure 3.13: Sequence diagram: asking the assistant a question

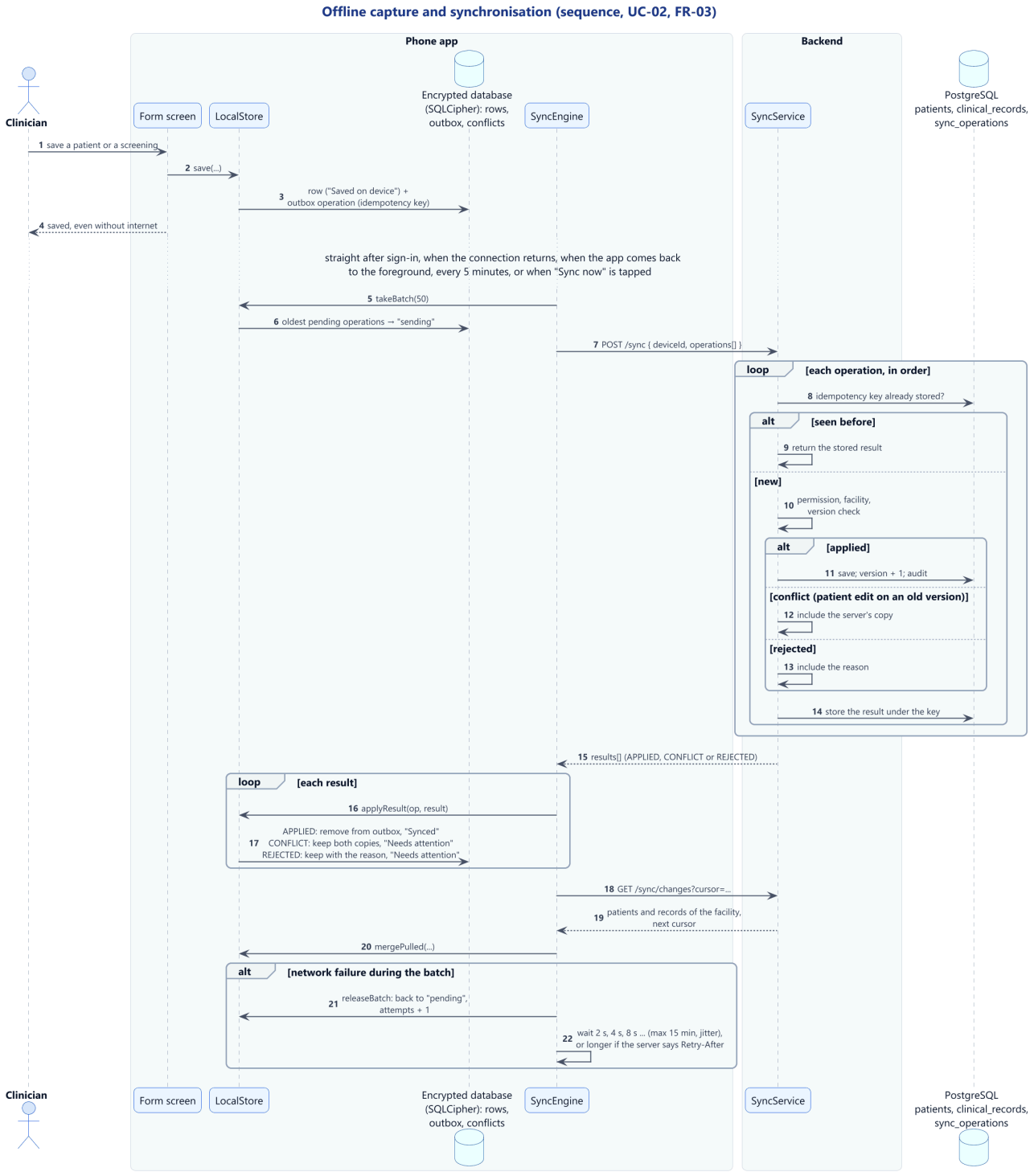
From the chat screen through the gateway guards, the chatbot service and the AI service to MongoDB and the audit log.



Files: PNG · SVG · source sequence-chat.puml

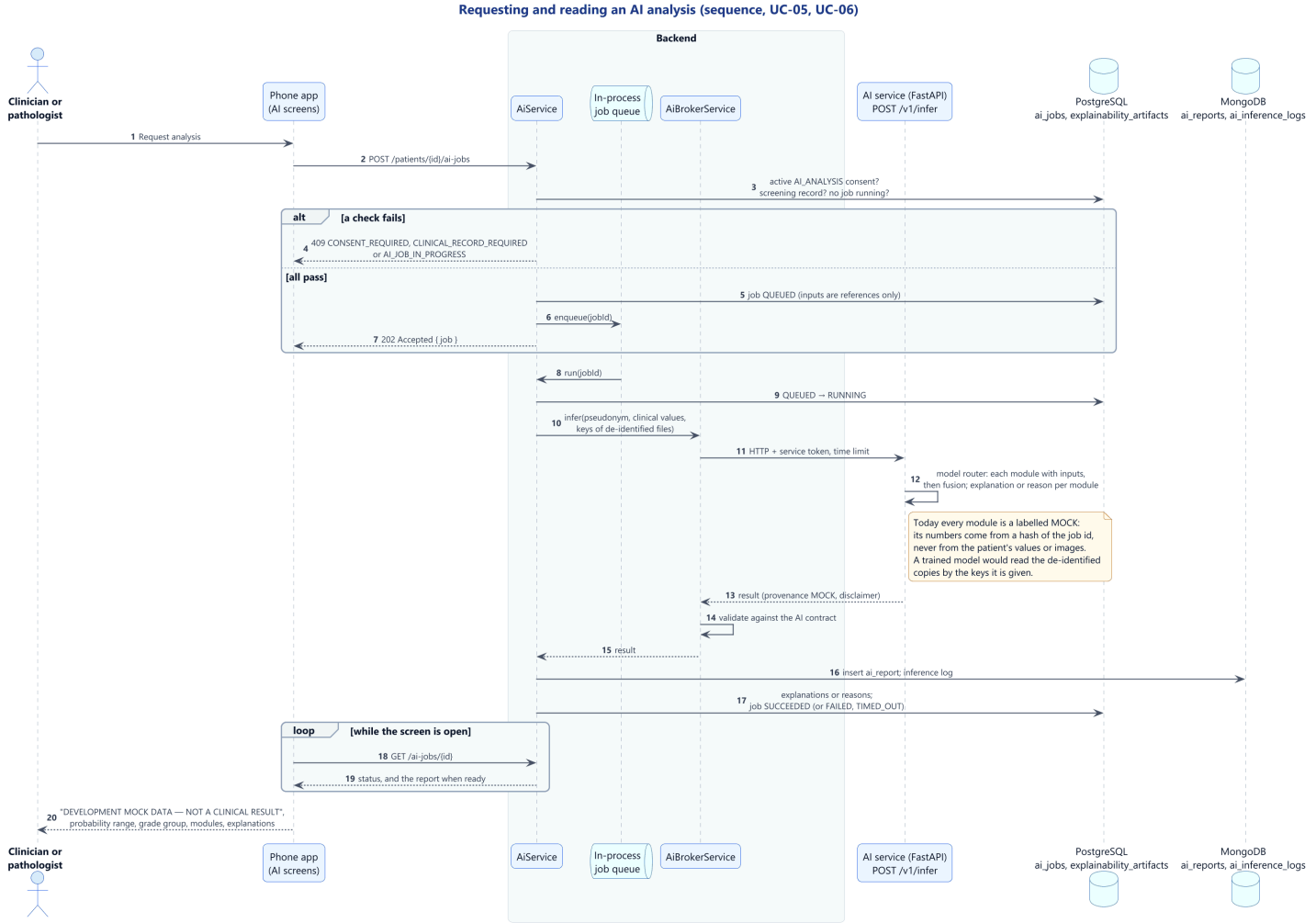
Figure 3.14: Sequence diagram: offline capture and synchronisation

The outbox on the phone, the batch sent to the server, the per-operation results, and downloading the facility's changes.



Files: PNG · SVG · source sequence-offline-sync.puml

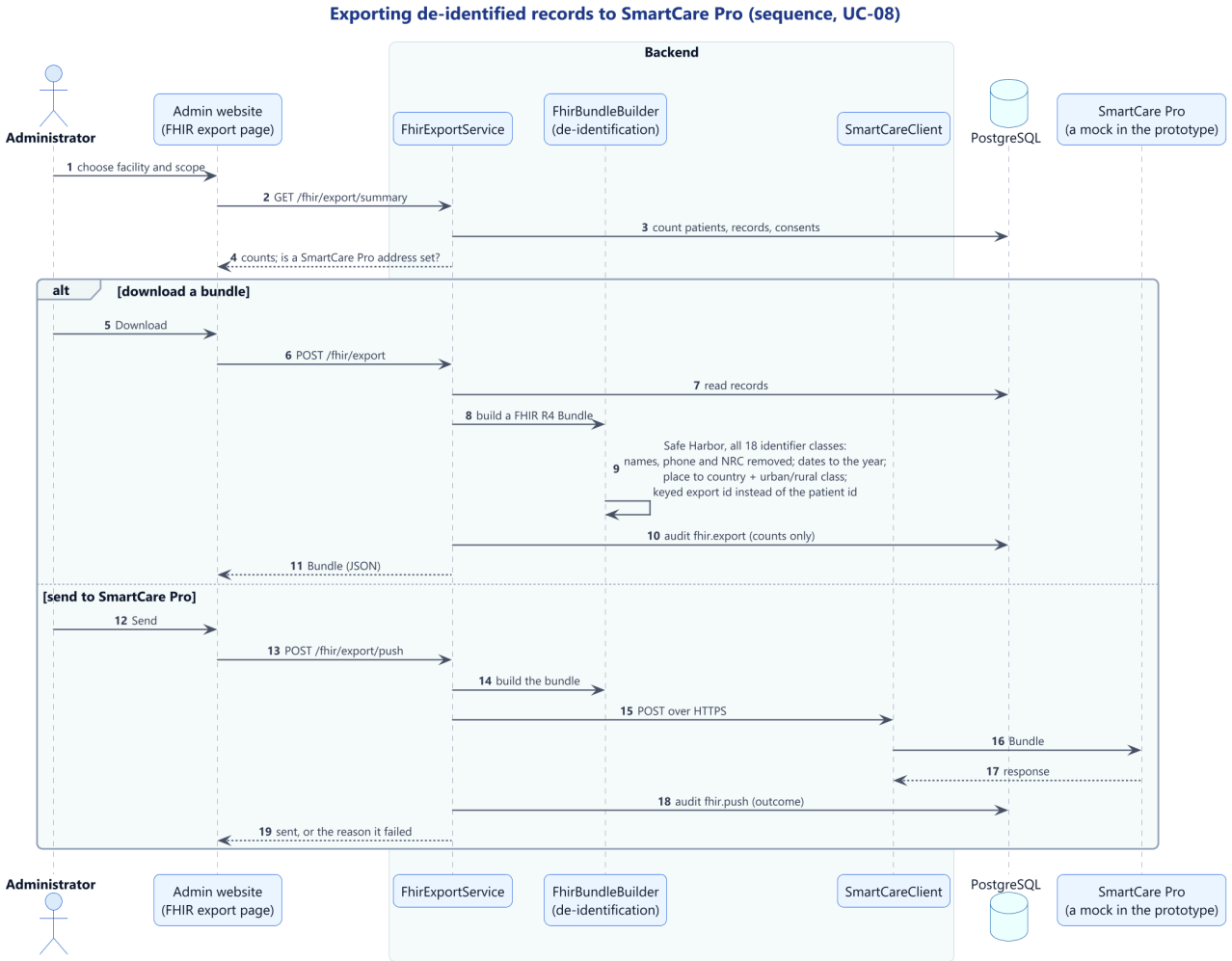
Figure 3.15: Sequence diagram: requesting and reading an AI analysis



Files: PNG · SVG · source sequence-ai-analysis.puml

Figure 3.16: Sequence diagram: exporting de-identified records to SmartCare Pro

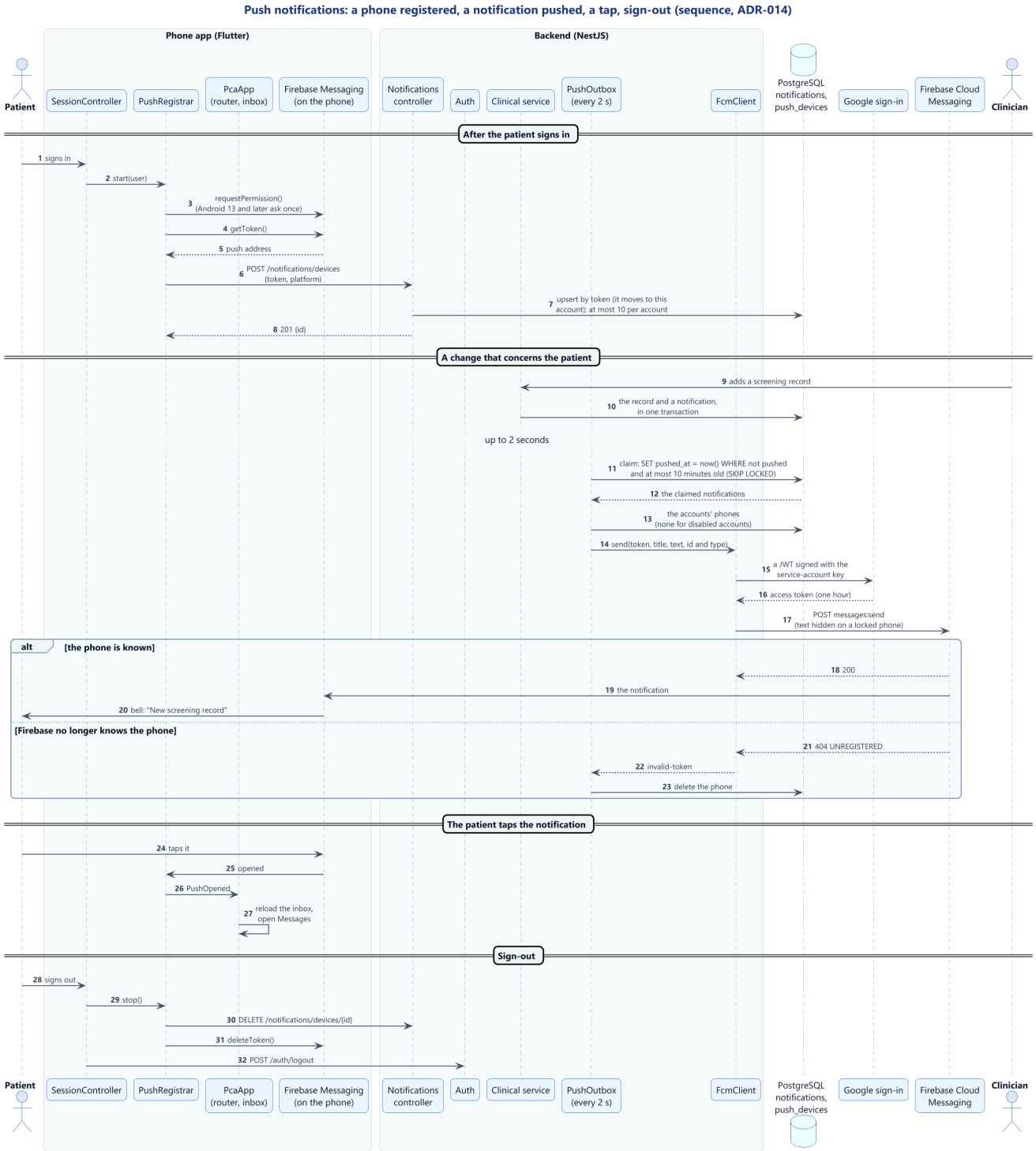
The FHIR R4 export (UC-08) with Safe Harbor de-identification of all 18 identifier classes.



Files: PNG · SVG · source sequence-fhir-export.puml

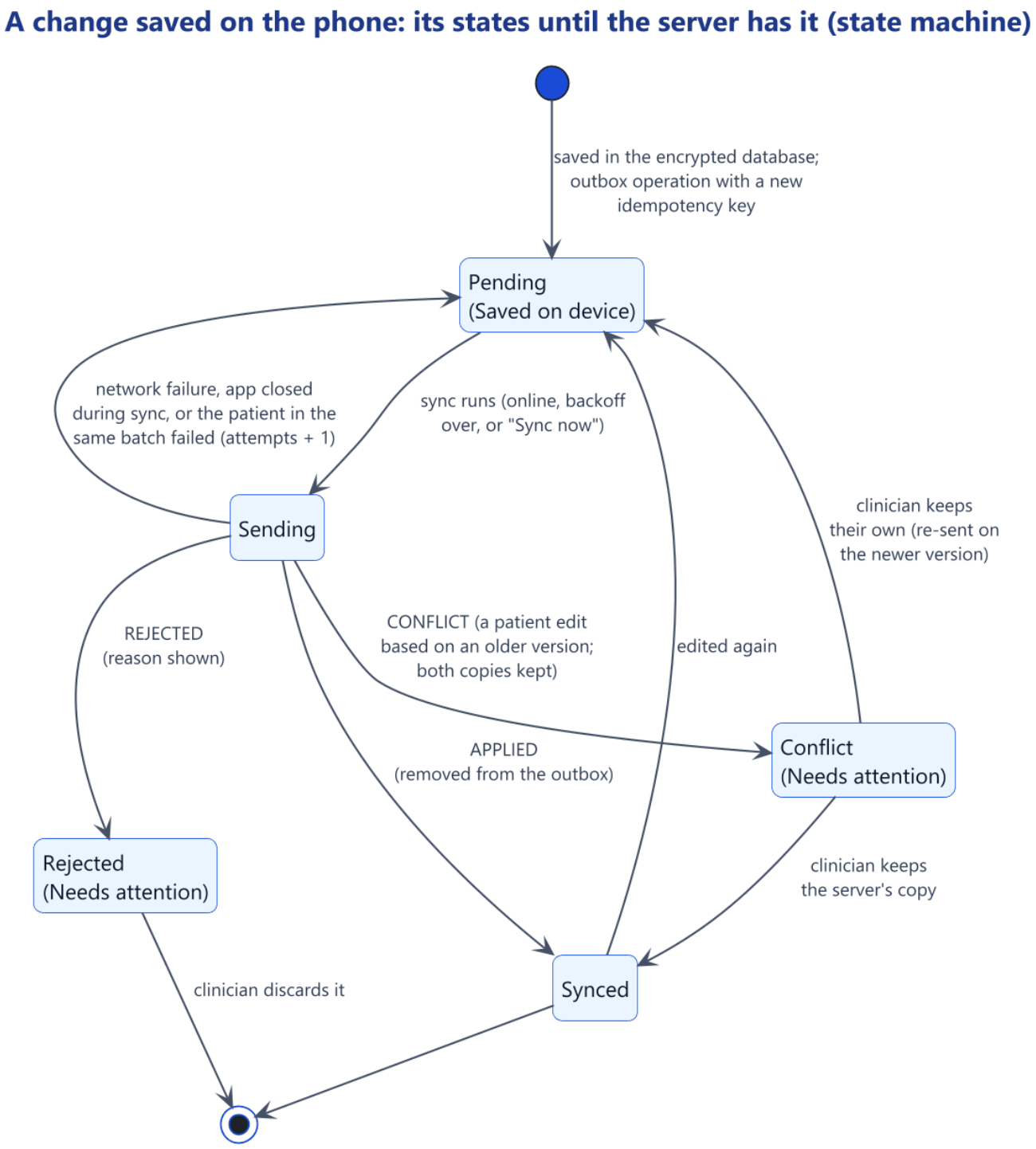
Figure 3.17: Sequence diagram: push notifications

A patient's phone is registered after sign-in; a new notification is claimed by the outbox and sent through Firebase Cloud Messaging (text only, hidden on a locked phone); a tap opens the messages; sign-out removes the phone (ADR-014). Push is switched on with the owner's Firebase project.



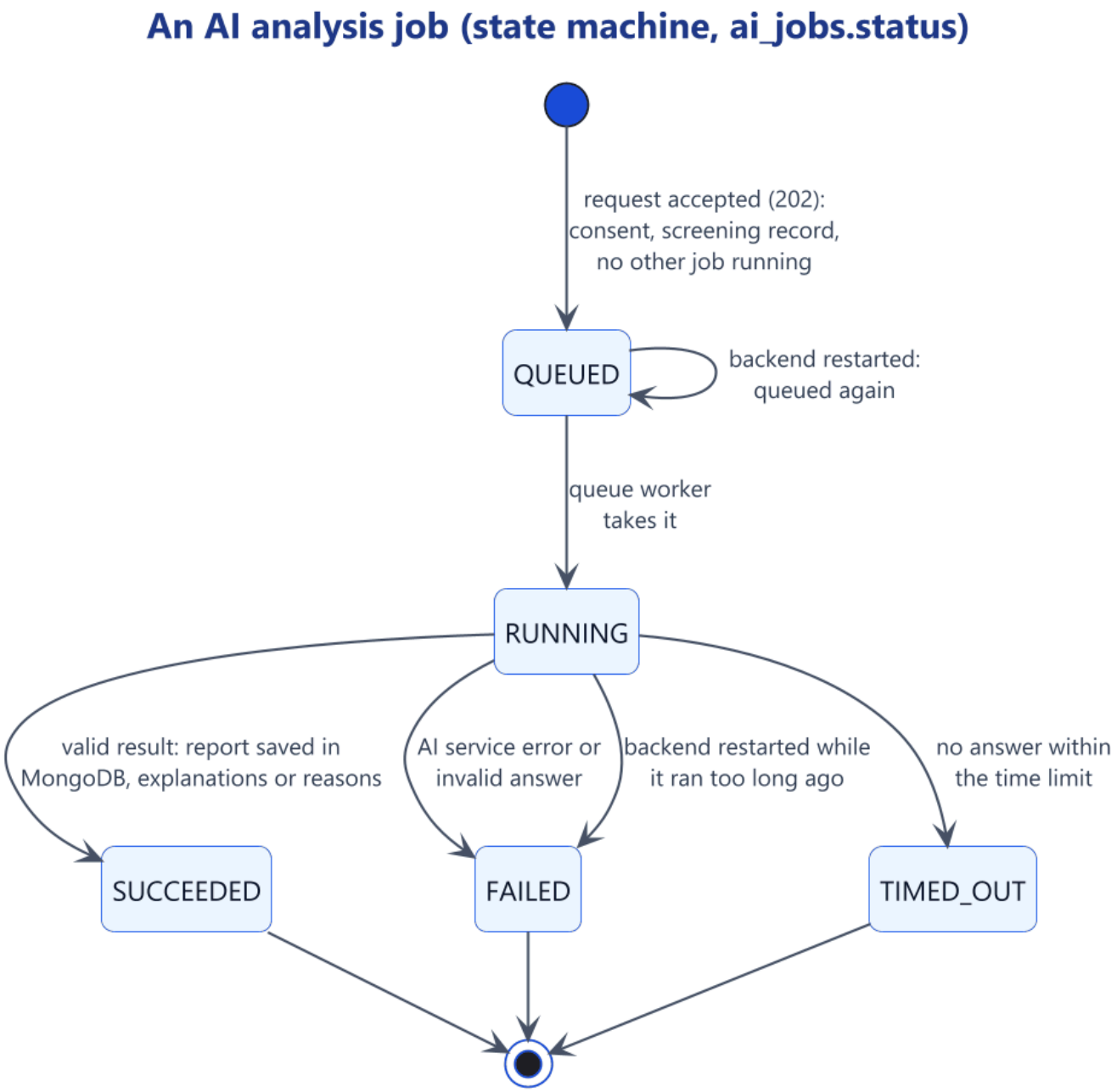
Files: PNG · SVG · source sequence-push.puml

Figure 3.18: State diagram: a change saved on the phone



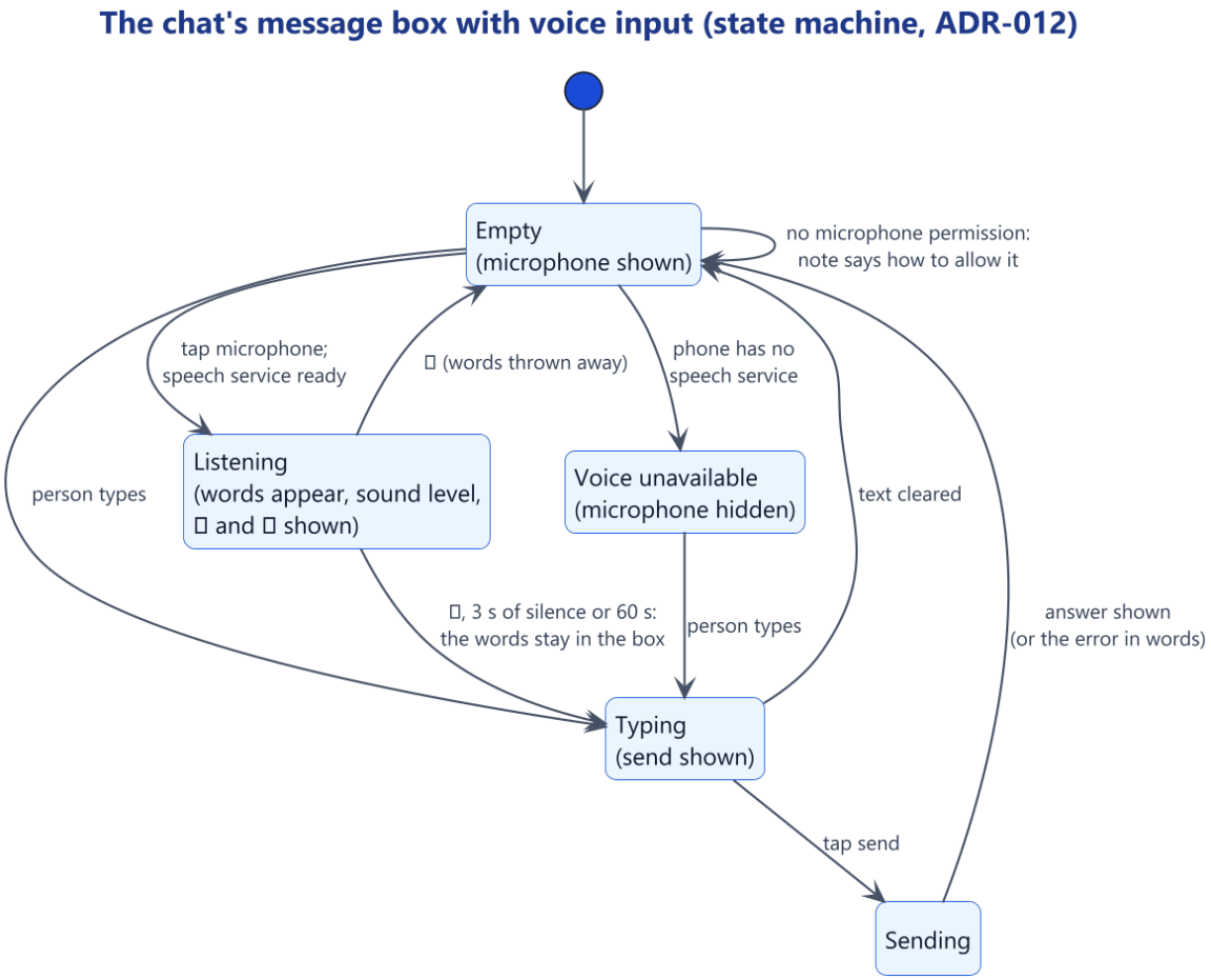
Files: PNG · SVG · source state-offline-change.puml

Figure 3.19: State diagram: an AI analysis job



Files: PNG · SVG · source state-ai-job.puml

Figure 3.20: State diagram: the chat's message box with voice input

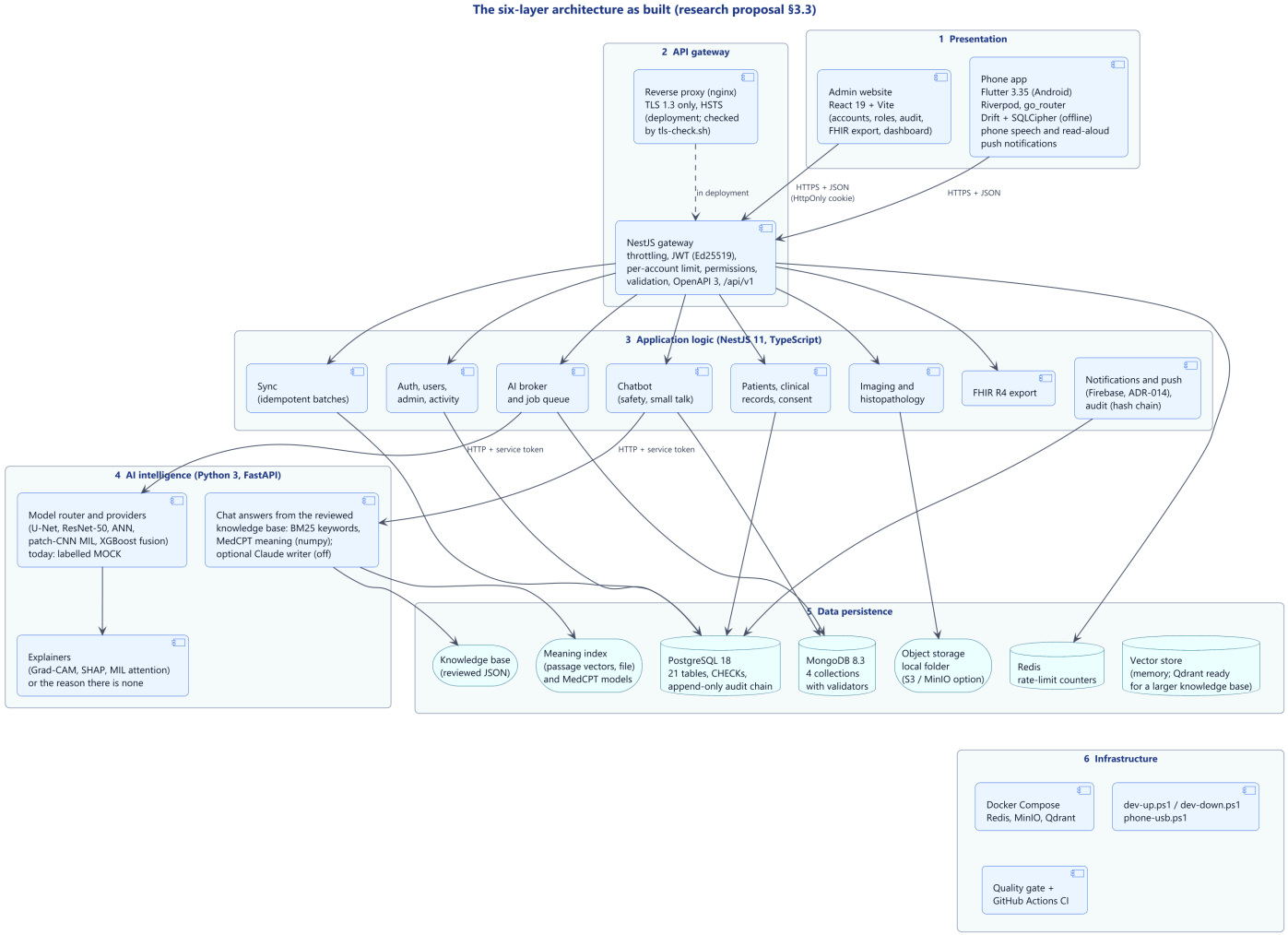


Files: PNG · SVG · source state-voice-input.puml

3.5 Program design, database design and interface design

Figure 3.21: The six-layer architecture as built

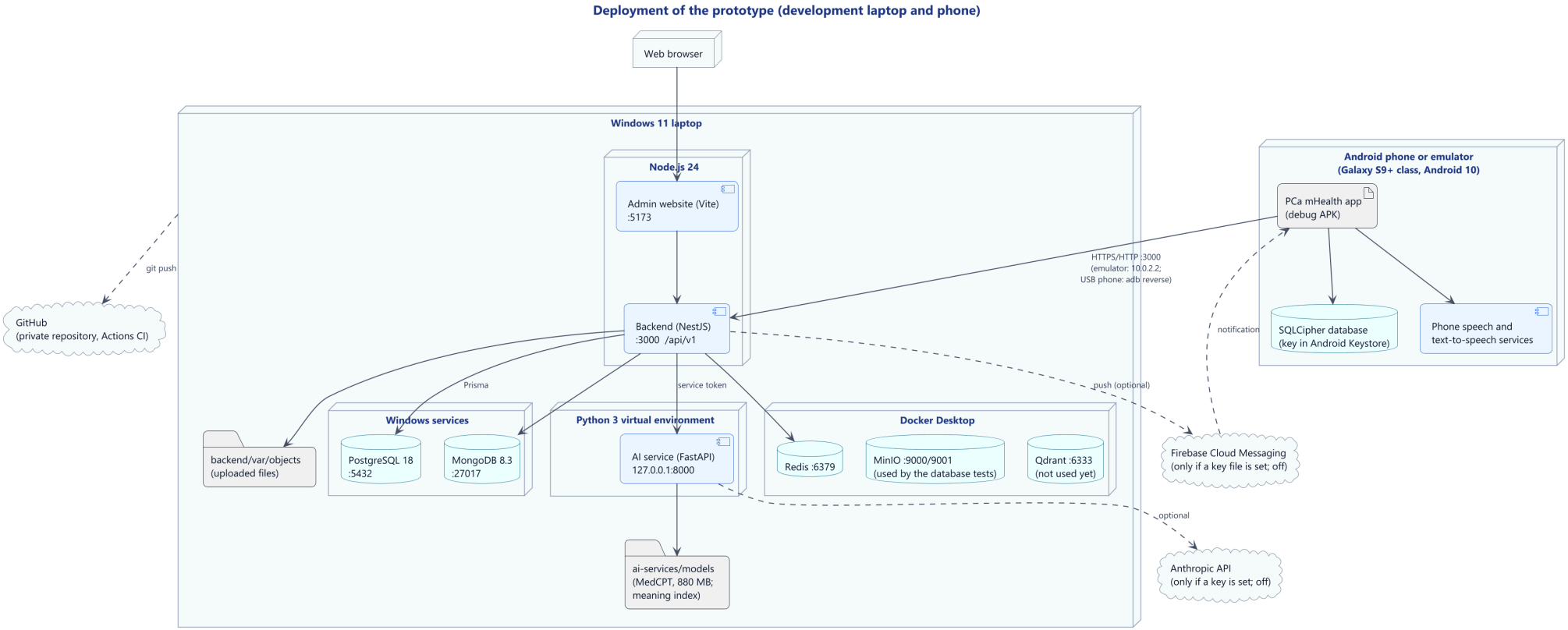
The proposal's six layers (§3.3) with what each contains in the prototype.



Files: PNG · SVG · source architecture-layers.puml

Figure 3.22: Deployment diagram

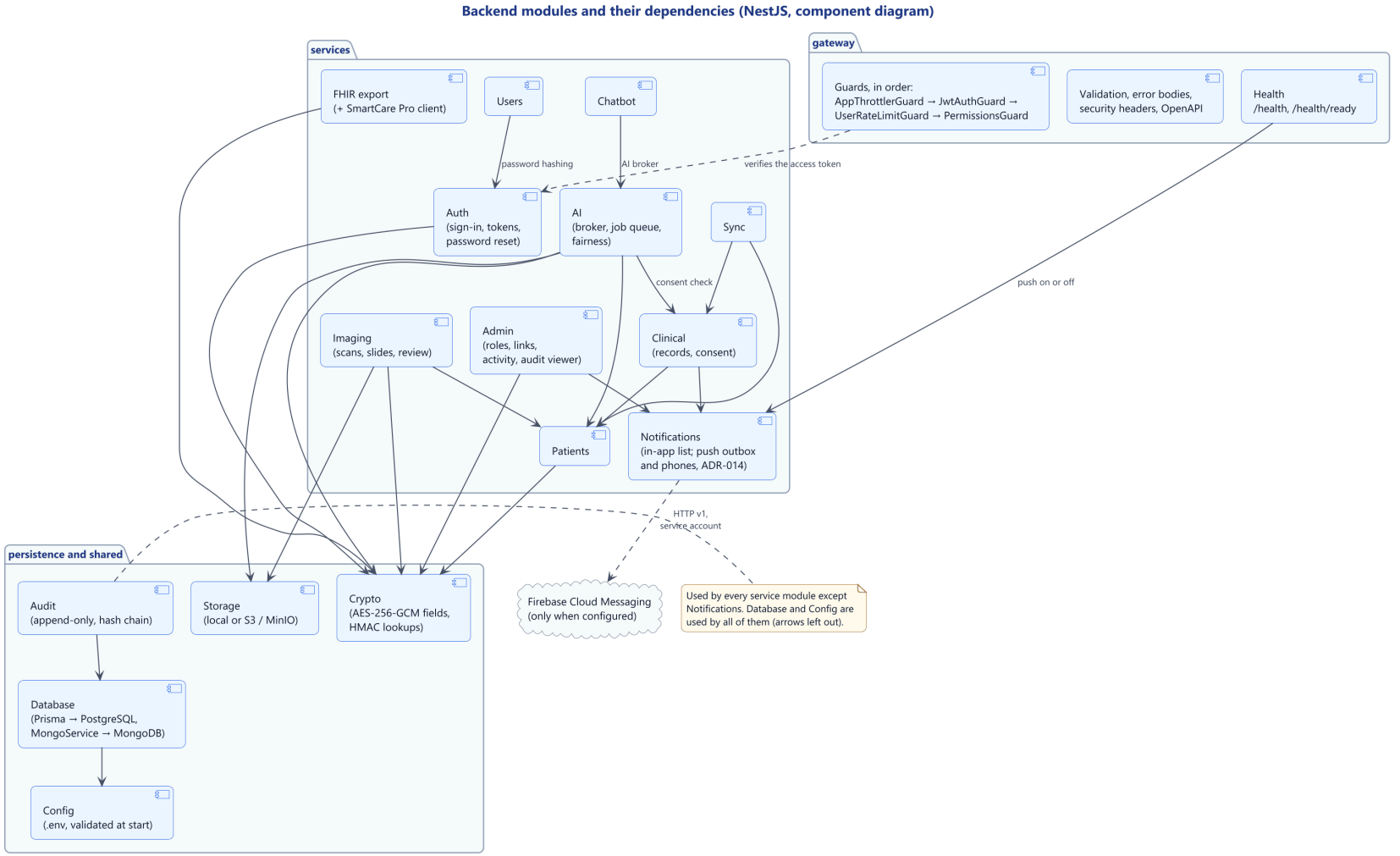
Where each part runs: the phone or emulator, the laptop's Windows services, Node.js, Python and Docker, and the outside services.



Files: PNG · SVG · source deployment.puml

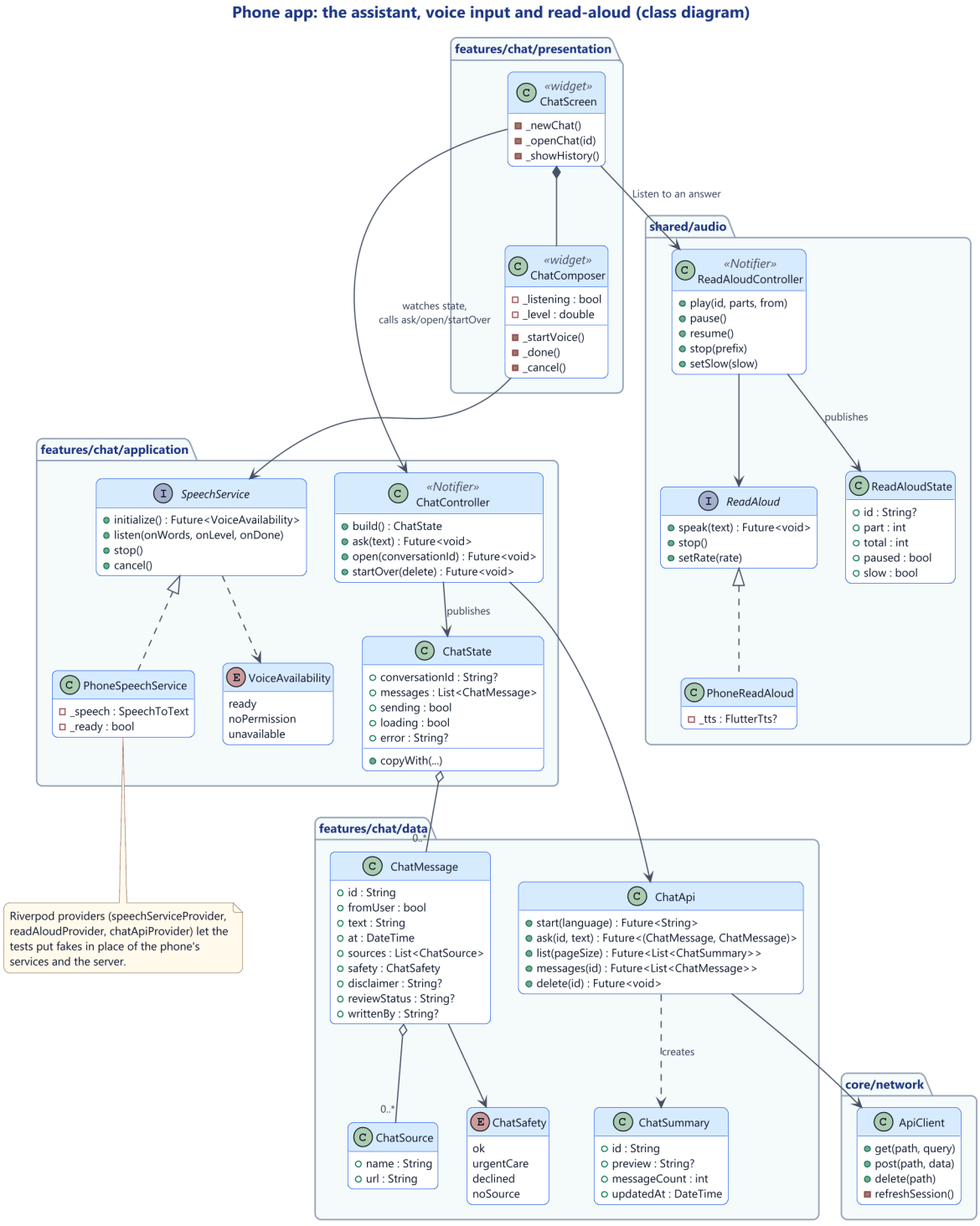
Figure 3.23: Component diagram: the backend modules

The NestJS modules and their real dependencies (taken from the services' constructors), and the order of the gateway guards.



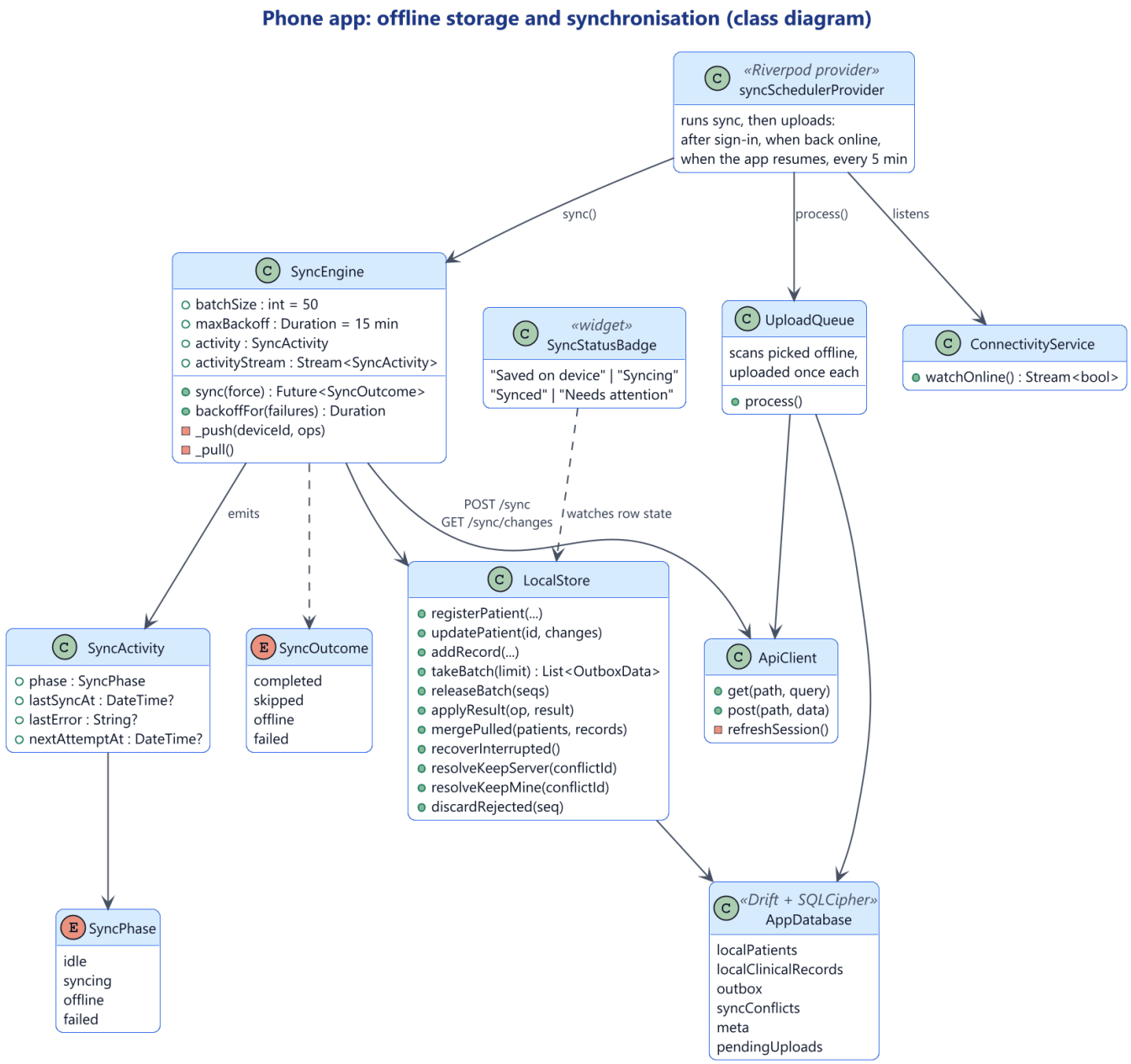
Files: PNG · SVG · source component-backend.puml

Figure 3.24: Class diagram: the app's assistant, voice input and read-aloud



Files: PNG · SVG · source class-app-chat-audio.puml

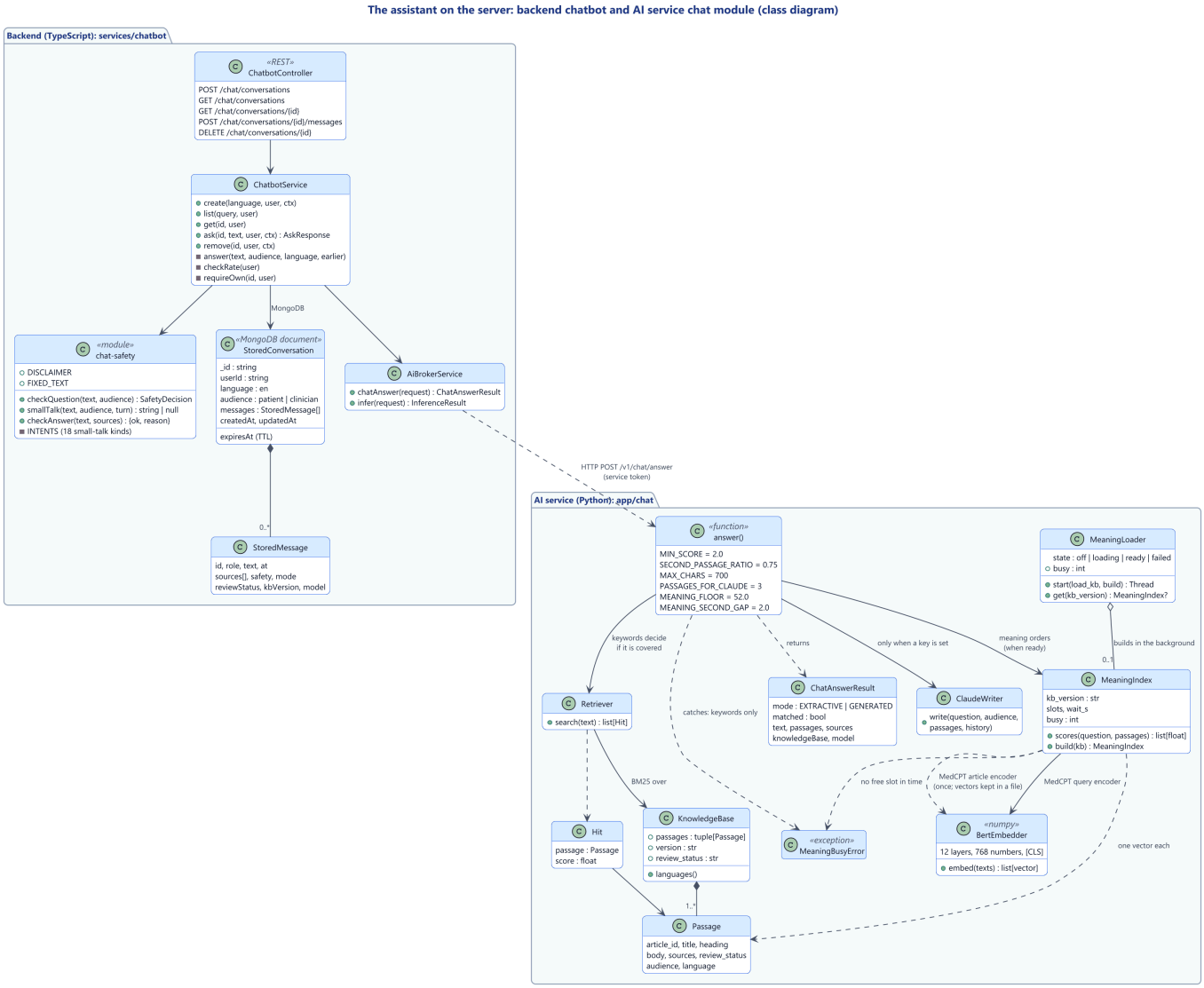
Figure 3.25: Class diagram: the app's offline storage and synchronisation



Files: PNG · SVG · source class-app-offline.puml

Figure 3.26: Class diagram: the assistant on the server

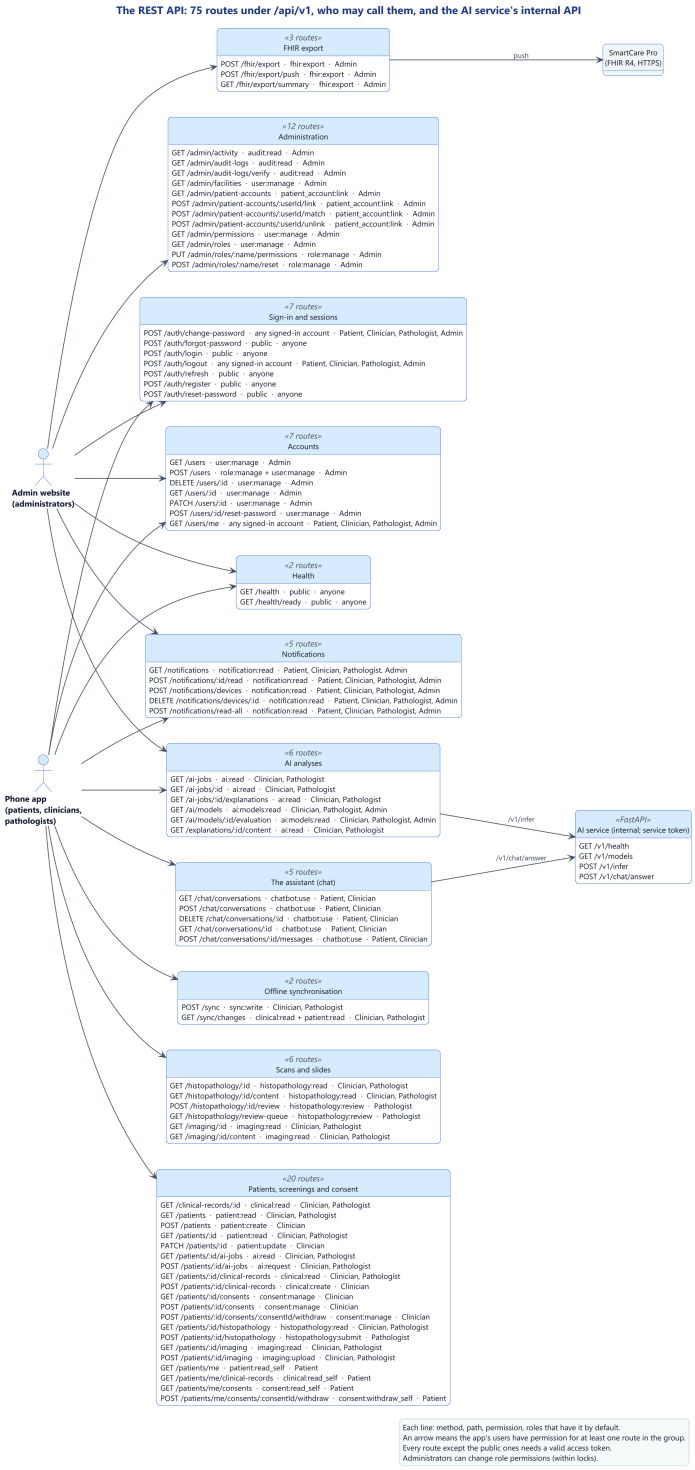
The backend's chatbot module and the AI service's chat module (keyword retrieval, the MedCPT meaning search, knowledge base, optional Claude writer).



Files: PNG · SVG · source class-chat-server.puml

Figure 3.27: API diagram: every route, its permission and its default roles

All REST routes under `/api/v1`, grouped by resource, with the apps whose users may call them, and the AI service's internal API. Generated from access-matrix.md.



Files: PNG · SVG · source generated: api-map.puml

Figure 3.28: The API explorer (OpenAPI 3, Swagger UI)

The backend describes its own API at `http://localhost:3000/api/docs` (screenshot of the running backend).

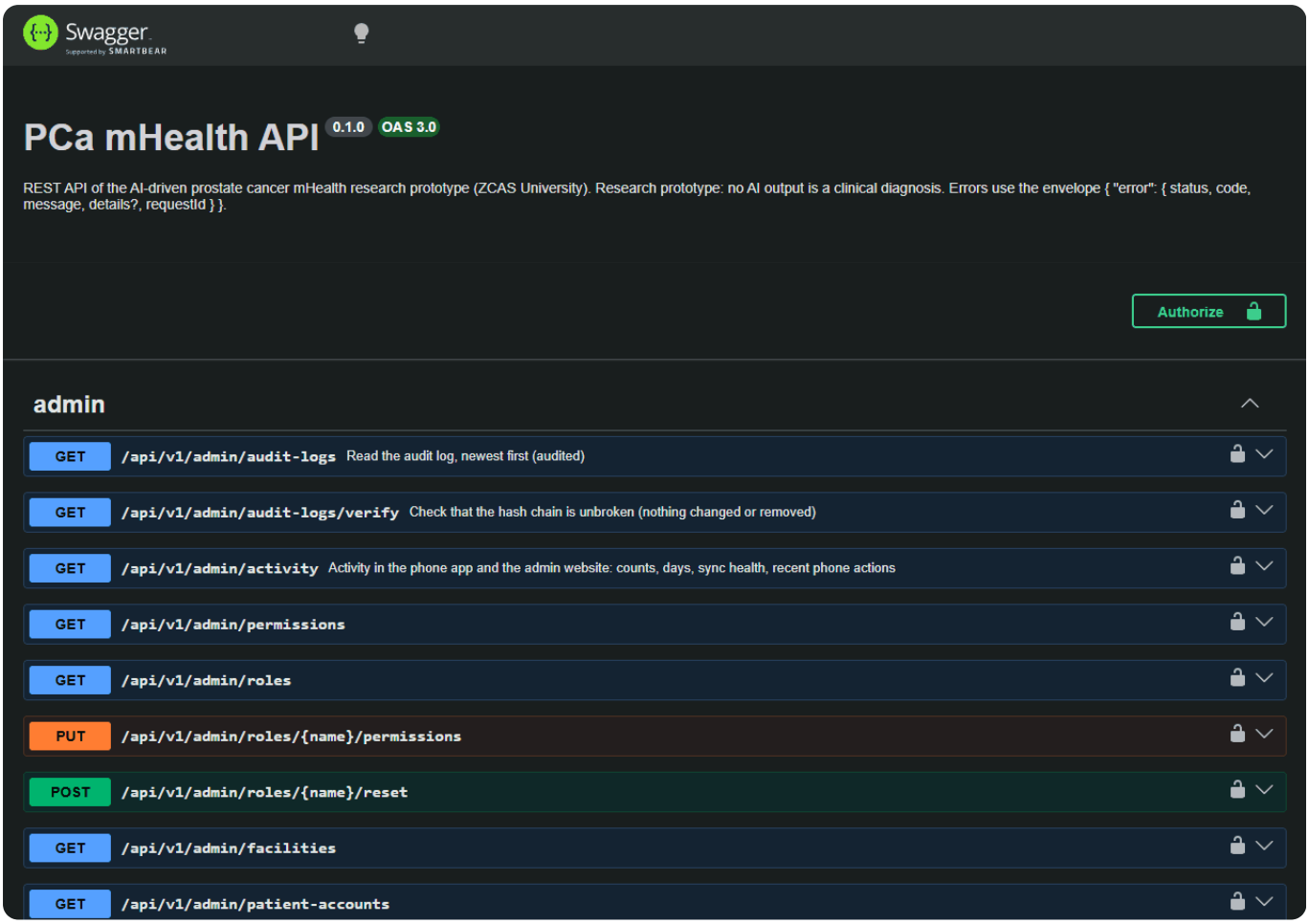
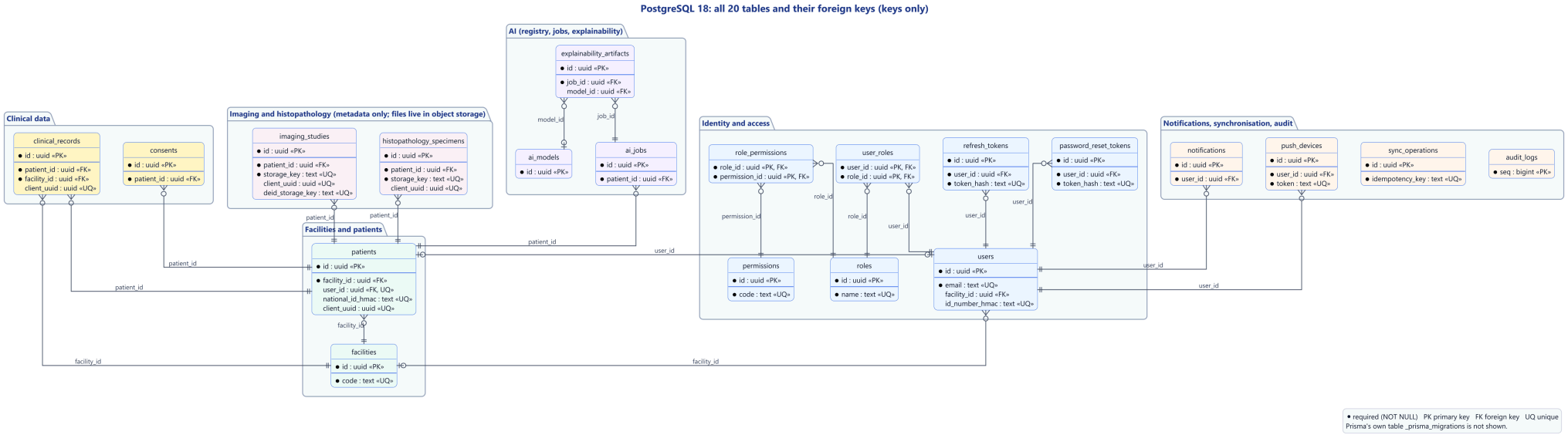


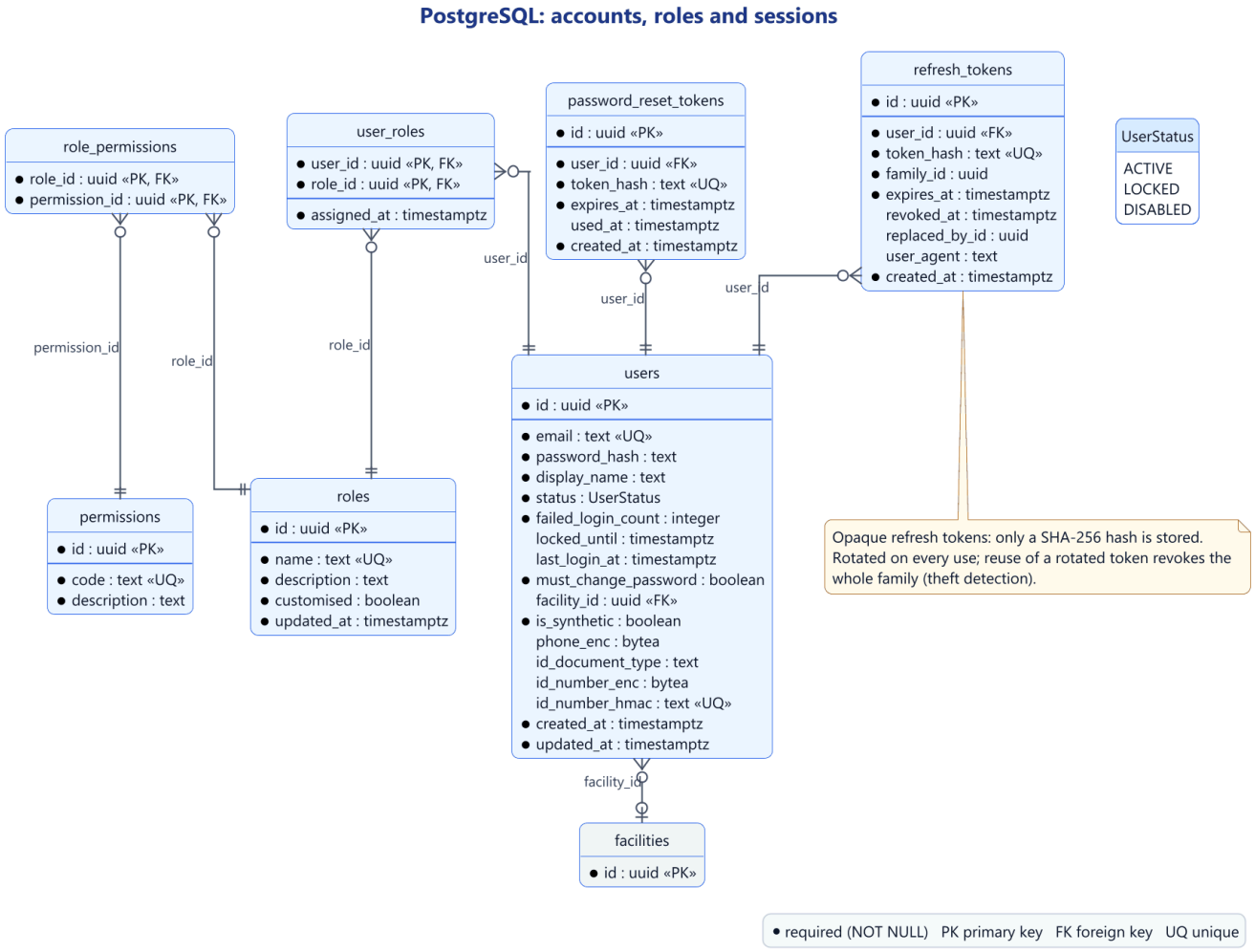
Figure 3.29: Entity-relationship diagram: all PostgreSQL tables and their keys

Generated from `schema.prisma`. Crow's-foot notation: a circle means "zero", a bar "one", a fork "many".



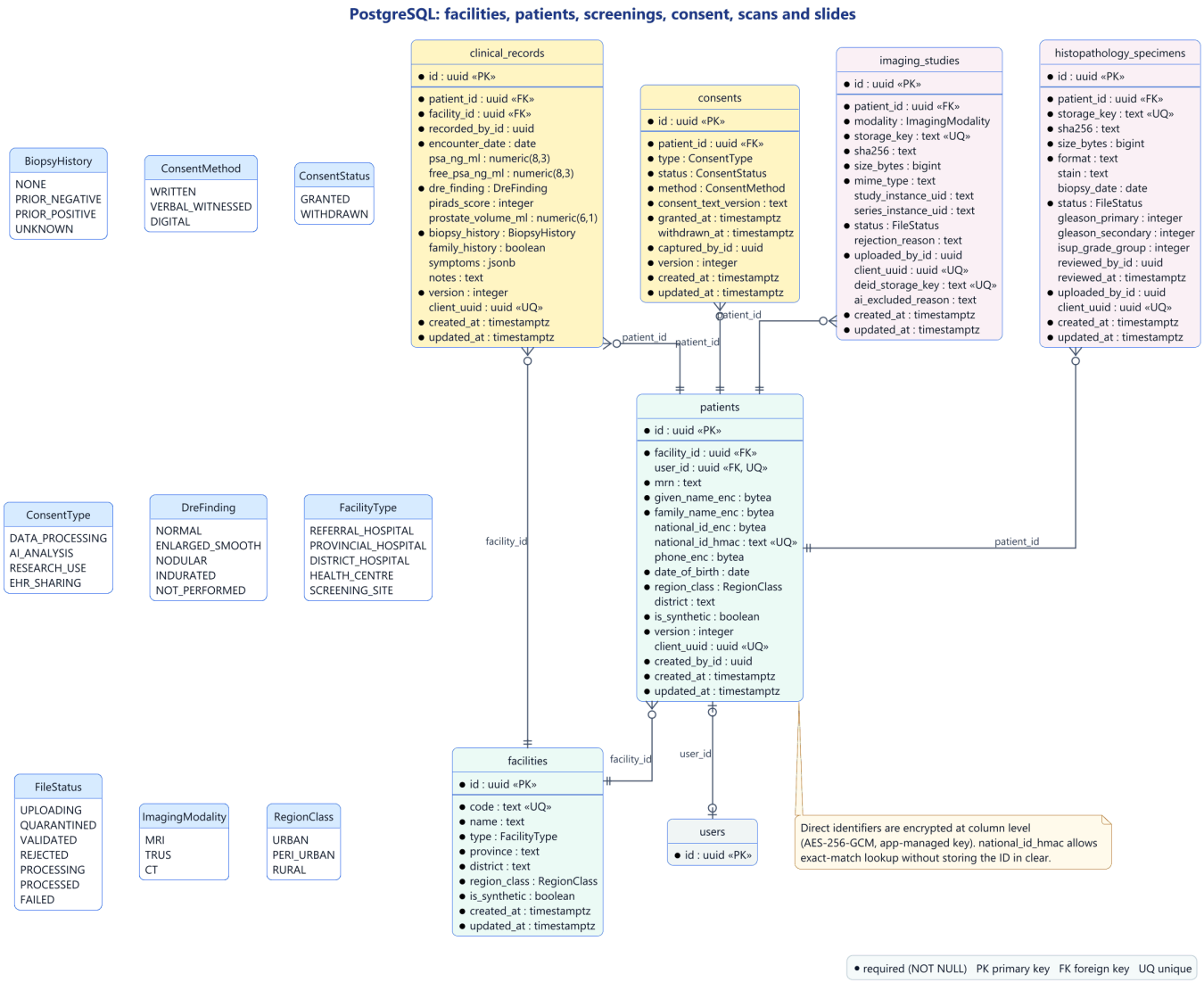
Files: PNG · SVG · source generated: erd-overview.puml

Figure 3.30: ERD: accounts, roles and sessions



Files: PNG · SVG · source generated: erd-accounts.puml

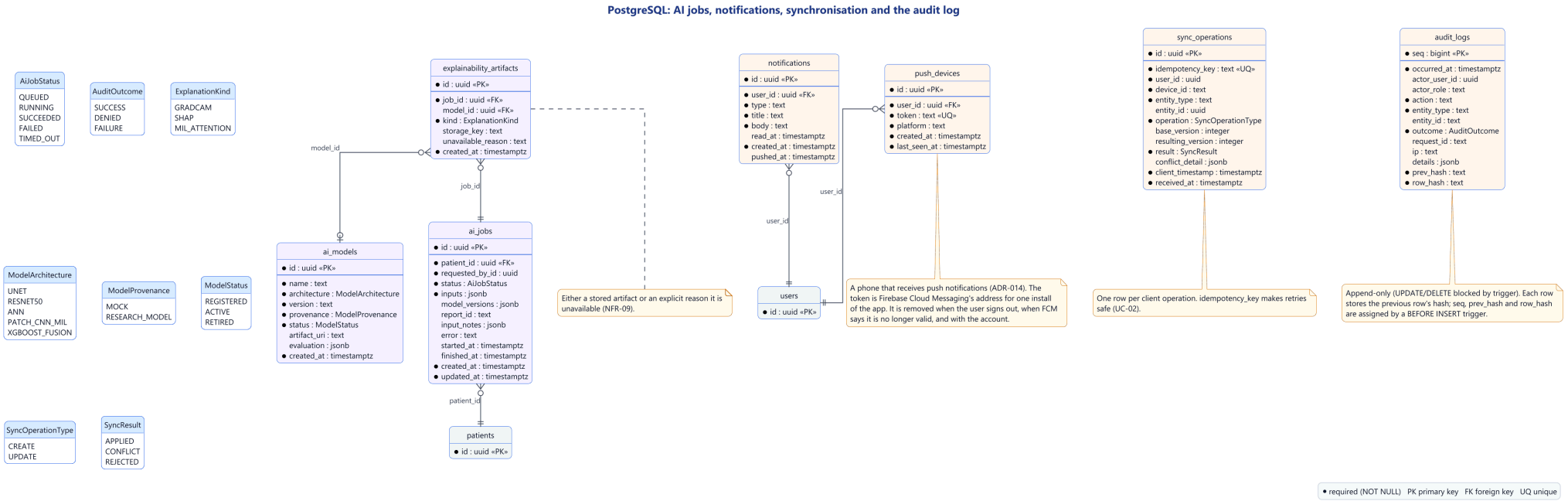
Figure 3.31: ERD: facilities, patients, screenings, consent, scans and slides



Files: PNG · SVG · source generated: erd-patients.puml

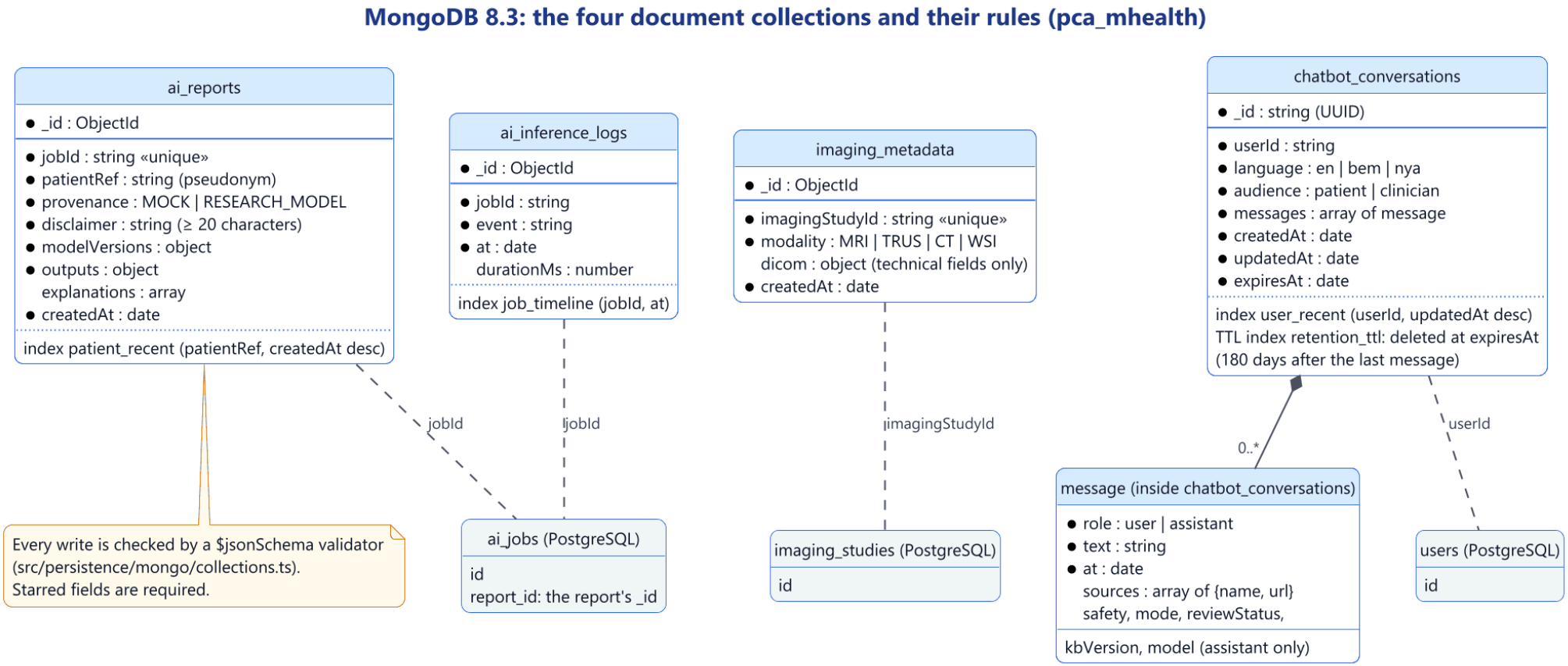
Figure 3.32: ERD: AI jobs, notifications, synchronisation and the audit log

Also the phones that receive push notifications (`push_devices` , ADR-014), and when each notification was pushed (`notifications.pushed_at`).



Files: PNG · SVG · source generated: erd-ai-sync-audit.puml

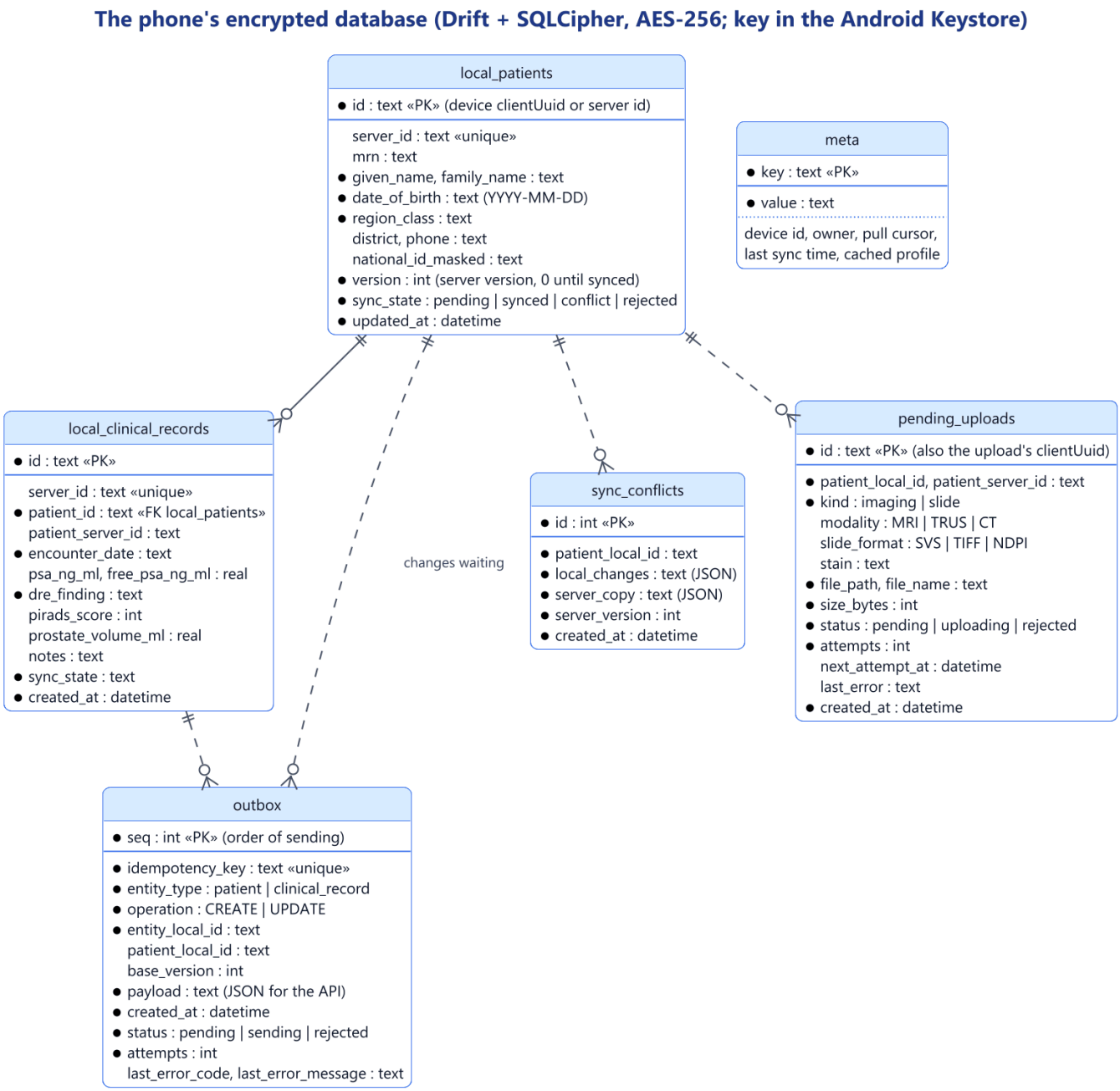
Figure 3.33: MongoDB: the four document collections and their rules



Files: PNG · SVG · source database-mongodb.puml

Figure 3.34: The phone's encrypted database

The six tables the app keeps on the phone (Drift with SQLCipher) for offline work.



Files: PNG · SVG · source database-phone.puml

Facts for the normalisation section

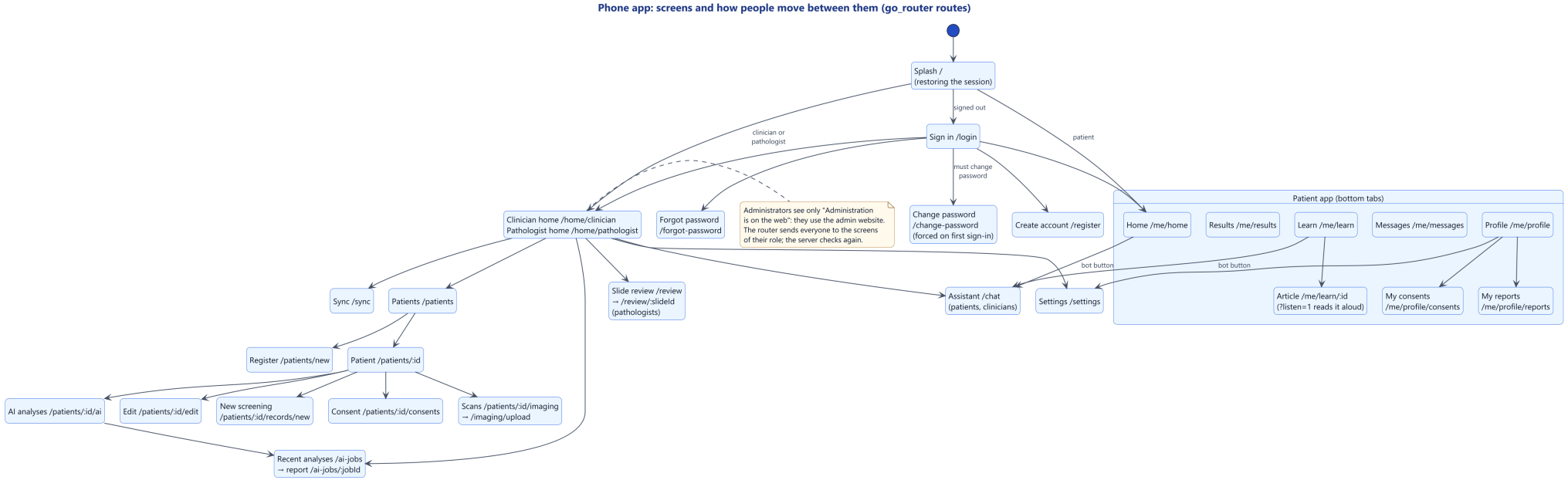
These are facts about the schema you can use when you discuss normalisation:

- **Keys:** every table has a primary key: a UUID `id`, except the two link tables `user_roles` (`user_id`, `role_id`) and `role_permissions` (`role_id`, `permission_id`), which have composite keys, and `audit_logs`, keyed by a sequence number `seq`.
- **Alternate keys** (unique columns): for example `users.email`, and `push_devices.token`, so one phone is registered only once.

- **Many-to-many relations** (users and roles, roles and permissions) are resolved by those two link tables.
- **Second normal form:** the only composite keys are those of the link tables; `role_permissions` has no other column, and `user_roles.assigned_at` depends on the whole key.
- **Third normal form:** facility details are stored once in `facilities` and referenced by `facility_id`; role names once in `roles`. PSA density and the free-to-total PSA ratio are calculated when a record is read, not stored.
- **Deliberate exceptions:**
- Some columns hold a JSON value: `clinical_records.symptoms`, `ai_jobs.inputs`, `model_versions` and `input_notes`, `ai_models.evaluation`, `sync_operations.conflict_detail`, `audit_logs.details`.
- `histopathology_specimens.isup_grade_group` is computed by the server from the two Gleason values and stored with the review.
- AI reports, chats and scan details live in MongoDB as documents, linked by id.
- **Integrity rules in the database:** 18 foreign keys, unique keys, CHECK constraints on clinical values (PSA, PI-RADS, Gleason, ISUP), and triggers that make `audit_logs` append-only and chain each row's hash to the previous one.

Figure 3.35: The app's screens and navigation

Every route of the app and how people move between screens, by role.



Files: PNG · SVG · source navigation-app.puml

Figure 3.36: Interface design: patient screens

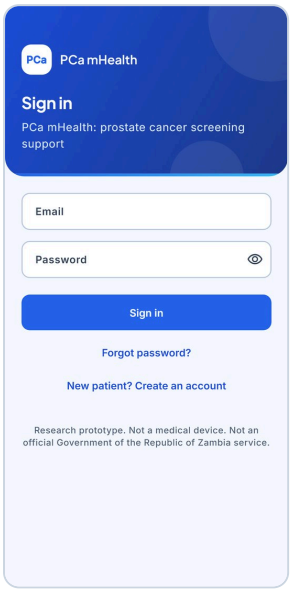
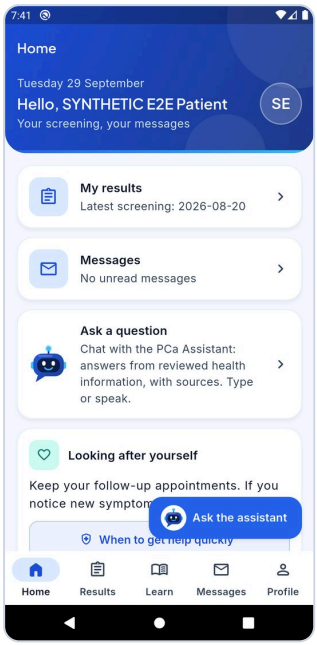
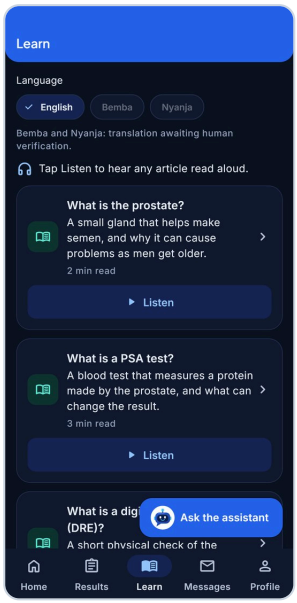
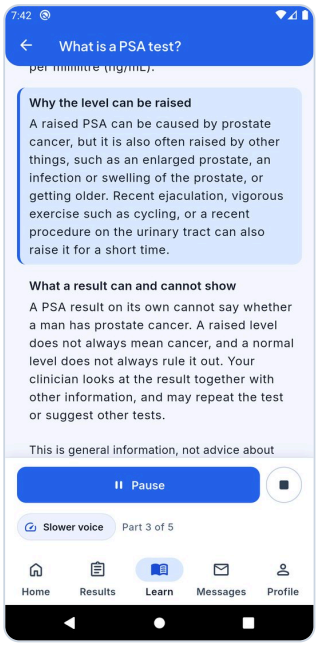
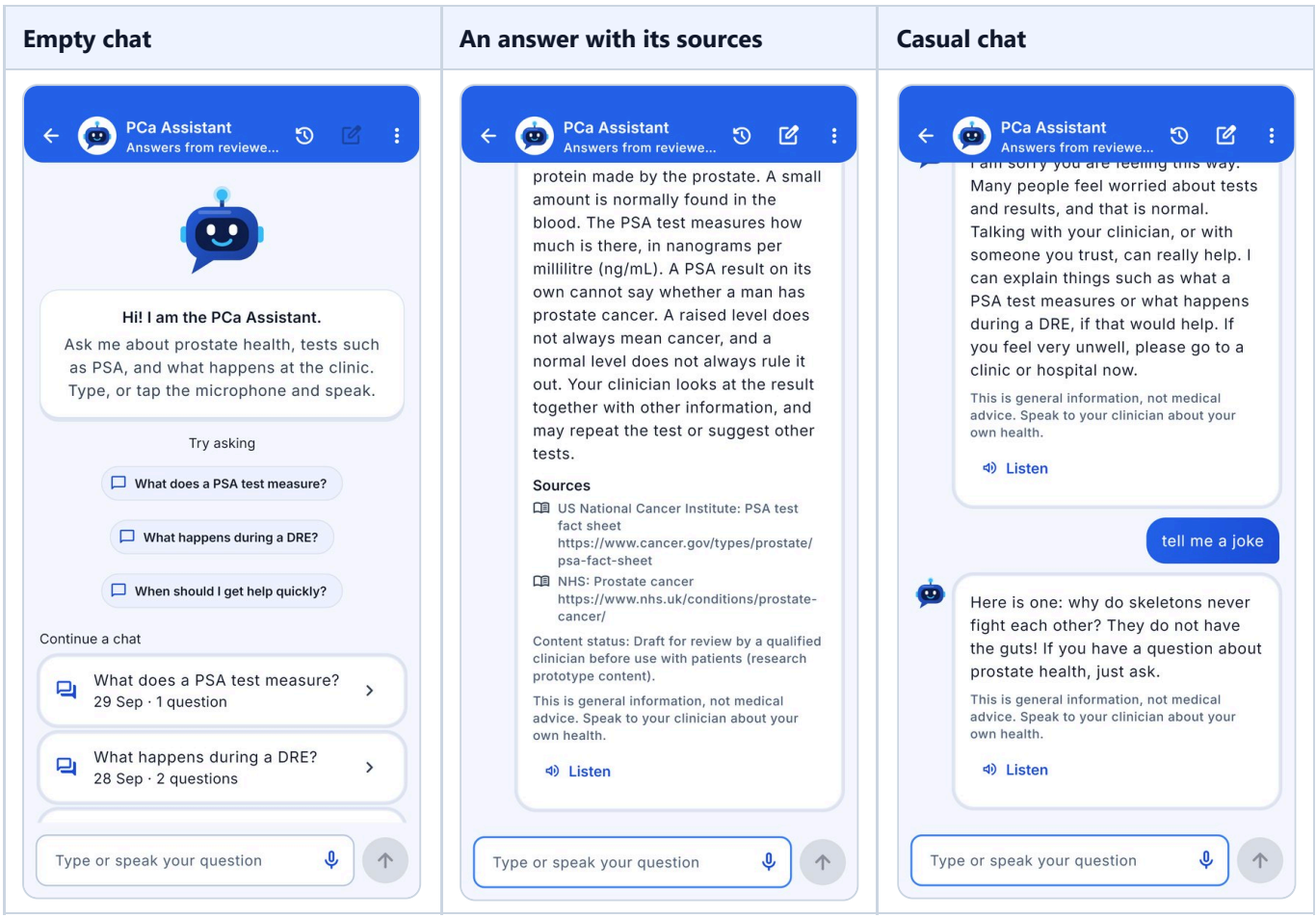
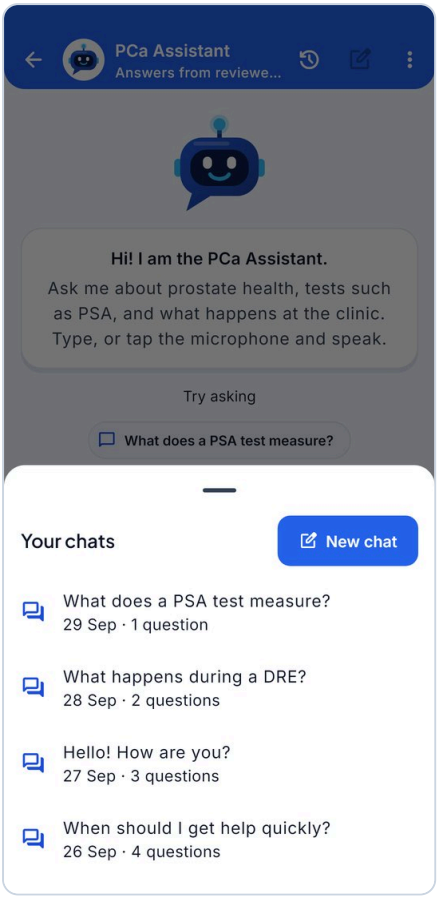
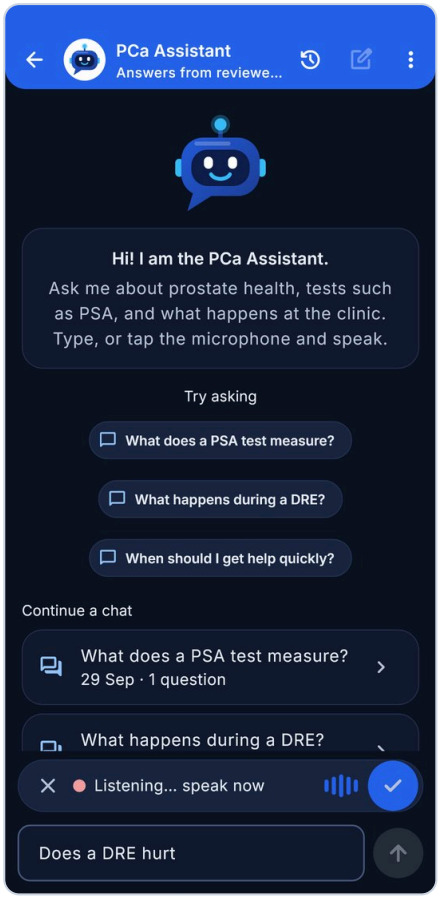
Sign-in	Patient home (emulator)	Learn (dark mode)	An article read aloud (emulator)
			

Figure 3.37: Interface design: the assistant



Past chats	Speaking a question
	

The answer is the AI service's real answer to that question; the casual replies are the backend's real replies; in the recording picture a stand-in plays the part of the phone's speech service.

Figure 3.38: Interface design: clinician home

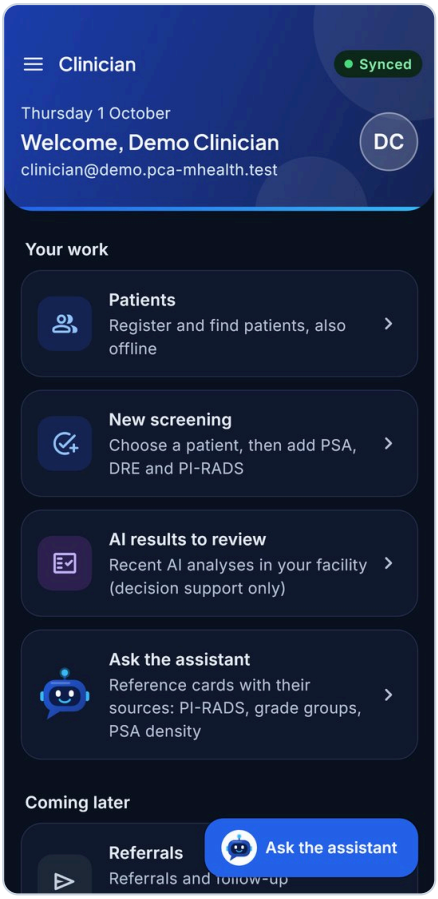
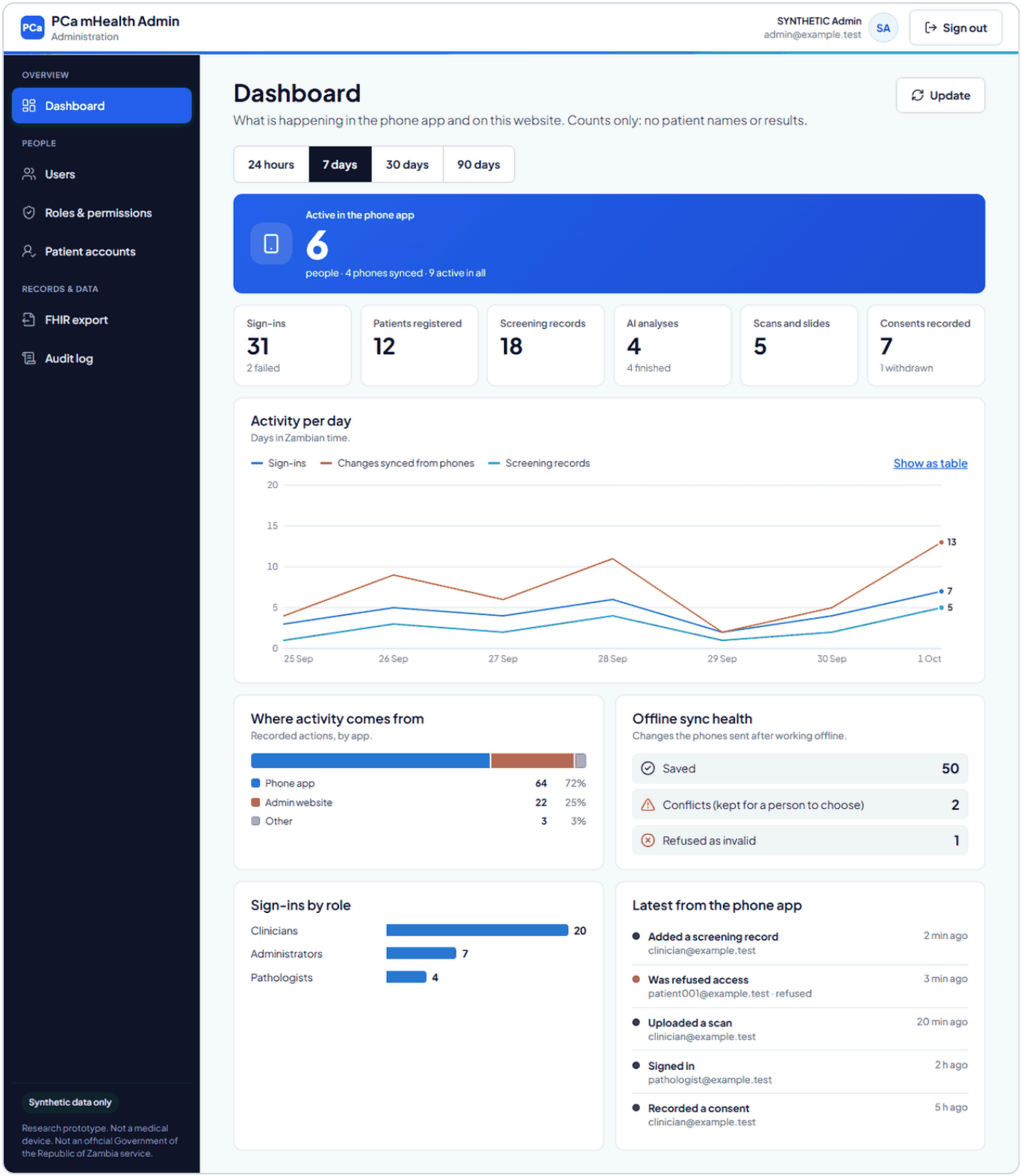
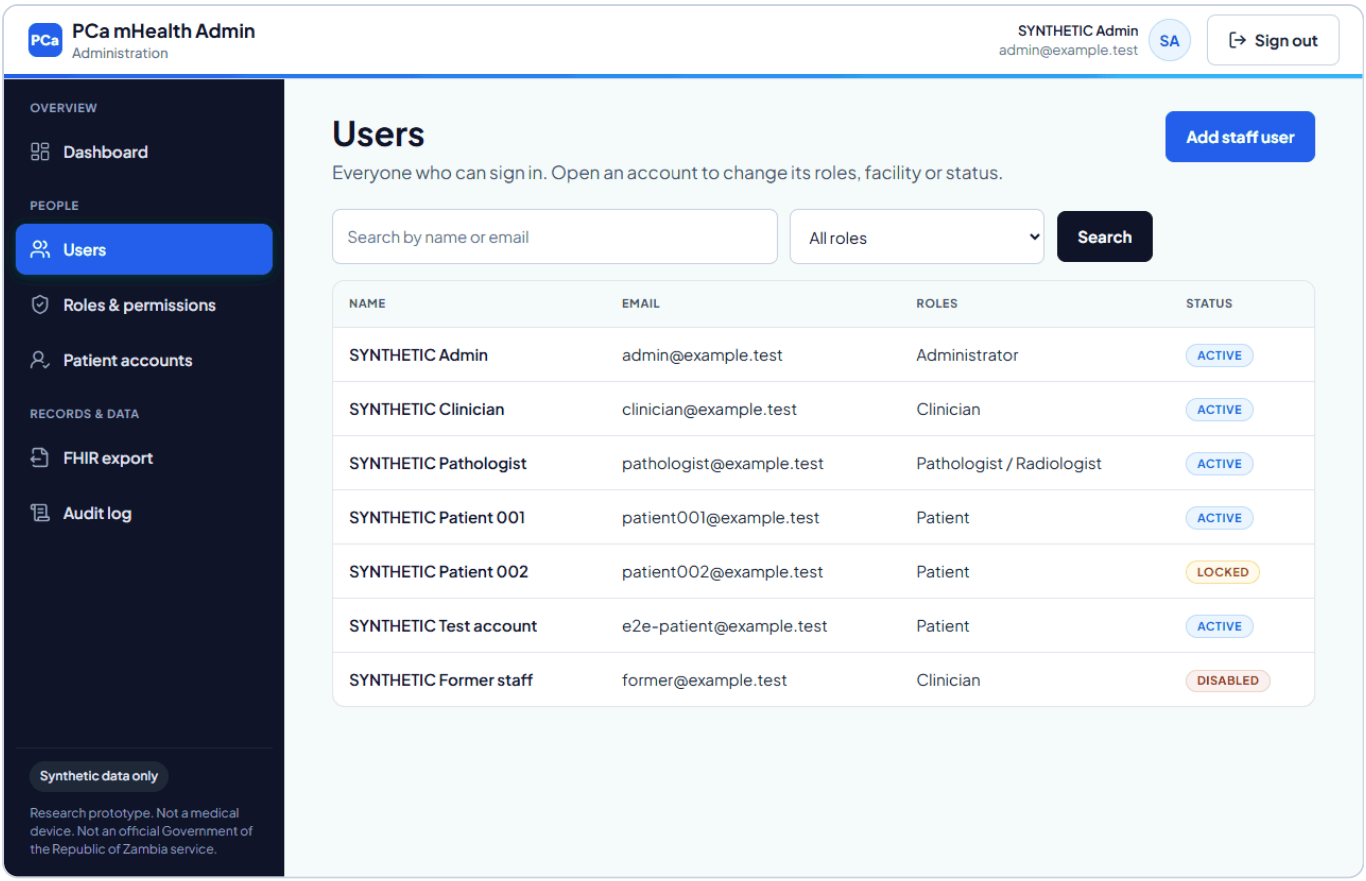
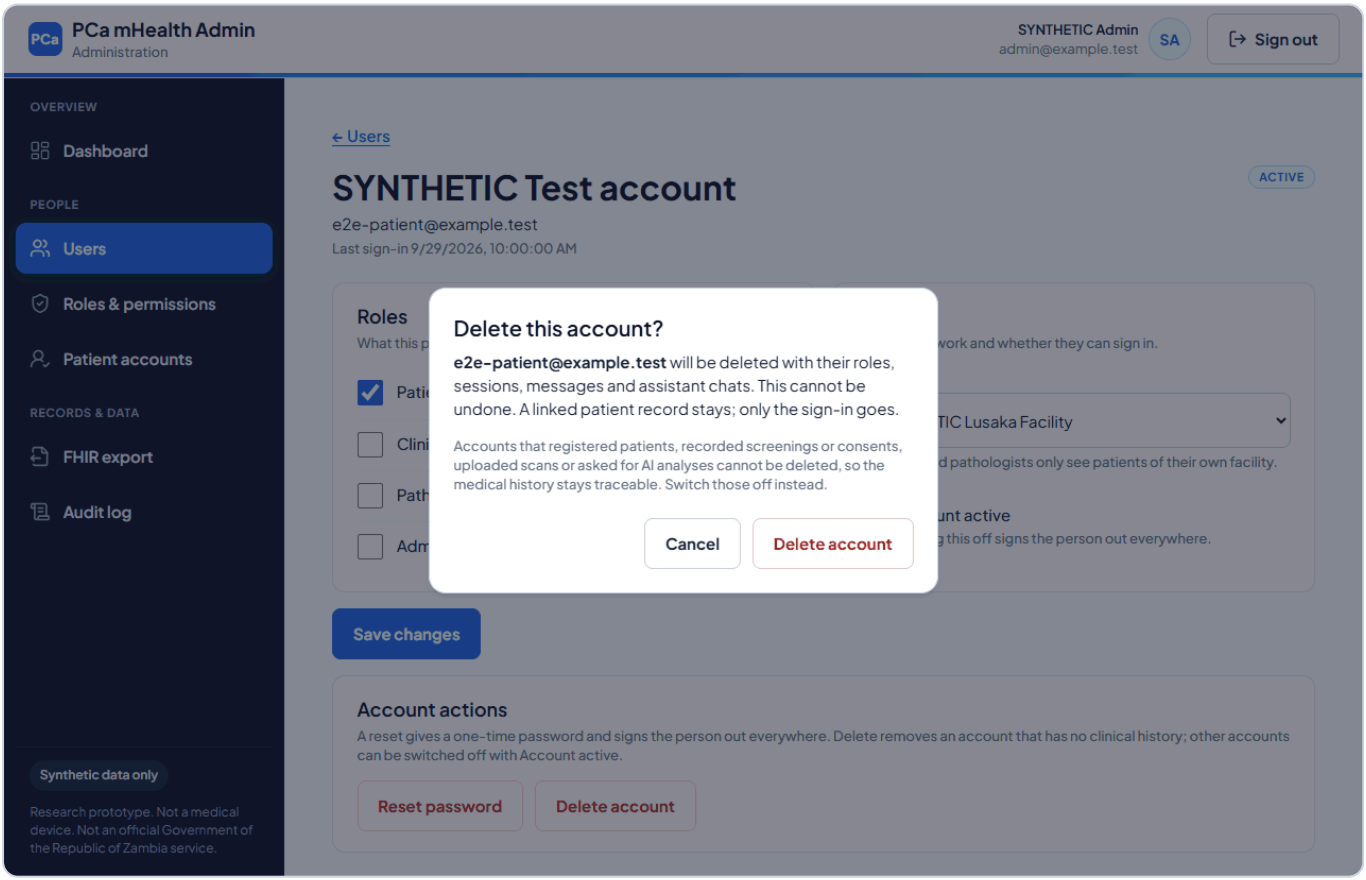


Figure 3.39: Interface design: the admin website

The dashboard, the accounts list and the delete dialog. Synthetic API answers were used for these pictures; the dashboard's numbers are made up for the picture.




PCa mHealth Admin
Administration
SYNTHETIC Admin
admin@example.test
SA
Sign out
OVERVIEW
Dashboard
PEOPLE
Users
Roles & permissions
Patient accounts
RECORDS & DATA
FHIR export
Audit log
Synthetic data only
Research prototype. Not a medical device. Not an official Government of the Republic of Zambia service.
Users
Everyone who can sign in. Open an account to change its roles, facility or status.
Add staff user
Search by name or email
All roles
Search
NAME
EMAIL
ROLES
STATUS
SYNTHETIC Admin
admin@example.test
Administrator
ACTIVE
SYNTHETIC Clinician
clinician@example.test
Clinician
ACTIVE
SYNTHETIC Pathologist
pathologist@example.test
Pathologist / Radiologist
ACTIVE
SYNTHETIC Patient 001
patient001@example.test
Patient
ACTIVE
SYNTHETIC Patient 002
patient002@example.test
Patient
LOCKED
SYNTHETIC Test account
e2e-patient@example.test
Patient
ACTIVE
SYNTHETIC Former staff
former@example.test
Clinician
DISABLED


PCa mHealth Admin
Administration
SYNTHETIC Admin
admin@example.test
SA
Sign out
OVERVIEW
Dashboard
PEOPLE
Users
Roles & permissions
Patient accounts
RECORDS & DATA
FHIR export
Audit log
Synthetic data only
Research prototype. Not a medical device. Not an official Government of the Republic of Zambia service.
← Users
SYNTHETIC Test account
ACTIVE
e2e-patient@example.test
Last sign-in 9/29/2026, 10:00:00 AM
Roles
Delete this account?
e2e-patient@example.test will be deleted with their roles, sessions, messages and assistant chats. This cannot be undone. A linked patient record stays; only the sign-in goes.
Accounts that registered patients, recorded screenings or consents, uploaded scans or asked for AI analyses cannot be deleted, so the medical history stays traceable. Switch those off instead.
Cancel
Delete account
Save changes
Account actions
A reset gives a one-time password and signs the person out everywhere. Delete removes an account that has no clinical history; other accounts can be switched off with Account active.
Reset password
Delete account

Chapter 4: System implementation and testing

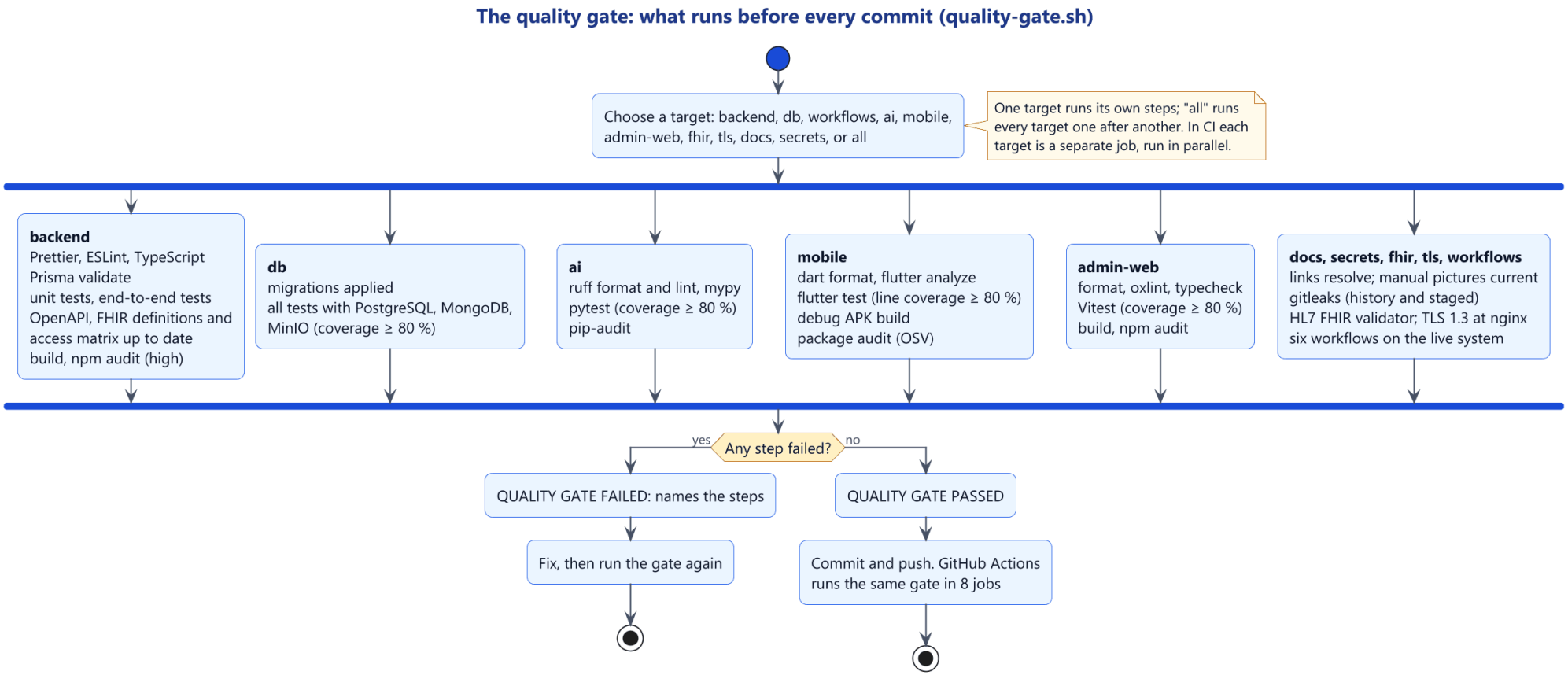
4.1 System guide and 4.2 Installation manual

The operations manual is the system guide and installation manual: what is installed, how to start, check and stop everything (§2 to §4), the phone app (§10) and the admin website (§11). how-to-test.md walks through every feature by hand. Figure 3.22 (deployment) shows where each part runs.

4.3 Testing plan and test output

Figure 4.1: The quality gate

What runs before every commit and in CI: formatting, lint, types, tests with at least 80 % coverage, builds and security audits, for each part.



Files: PNG · SVG · source activity-quality-gate.puml

Figure 4.2: Test output: the backend gate

The real output of the backend gate (summary lines). The testing strategy is in testing-strategy.md; the counts for every part are in §12 of the operations manual.

The quality gate for the backend

```
PS D:\Final Year Project\pca-mhealth> powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 backend
(only the summary lines are shown)
==> backend
    PASS: backend format
    PASS: backend lint
    PASS: backend typecheck
    PASS: prisma validate
Tests:       323 passed, 323 total
    PASS: backend unit tests
Tests:       20 passed, 20 total
    PASS: backend e2e tests
    PASS: openapi document up to date
    PASS: FHIR definitions up to date
    PASS: access matrix up to date and within the review rules
    PASS: backend build
    PASS: backend npm audit
QUALITY GATE PASSED (backend)
```

The user acceptance test (proposal §3.8.2) has its own kit in uat/: the plan, the information sheet and consent form, task sheets for each role, the SUS questionnaire and a results template whose tables fit this section. It may only be used after ethics approval.

4.4 Main function codes

The code that meets each objective of the proposal. Every picture shows the real file with its line numbers.

Objective 1: a secure, offline-first mobile application (AES-256, TLS 1.3, RBAC, FHIR R4)

Figure 4.3: The phone's encrypted database (SQLCipher key in the Keystore)

1-presentation-layer/mobile-app/lib/core/db/open_database.dart lines 16–57

```dart
/// Opens the on-device database, encrypted with SQLCipher (AES-256, NFR-01).
/// The 256-bit key is random per installation and kept only in the Android
/// Keystore / iOS Keychain. If the key is lost (e.g. app data restored without
/// the Keystore), the unreadable file is replaced by an empty database; the
/// data can be downloaded again from the server.
Future<AppDatabase> openAppDatabase({FlutterSecureStorage? storage}) async {
  final secure = storage ?? const FlutterSecureStorage();
  final dir = await getApplicationSupportDirectory();
  final file = File(p.join(dir.path, _fileName));

  var key = await secure.read(key: _keyName);
  if (key == null) {
    if (await file.exists()) await file.delete();
    key = _randomHexKey();
    await secure.write(key: _keyName, value: key);
  }
  final hexKey = key;

  return AppDatabase(
    NativeDatabase.createInBackground(
      file,
      isolateSetup: useSqlCipher,
      setup: (db) {
        // Fails fast if plain SQLite was loaded instead of SQLCipher.
        final version = db.select('PRAGMA cipher_version;');
        if (version.isEmpty) {
          throw StateError(
            'SQLCipher is not available; refusing to store data unencrypted',
          );
        }
        db.execute("PRAGMA key = \"x'$hexKey'\";");
        // Reading the schema verifies the key.
        db.select('SELECT count(*) FROM sqlite_master;');
      },
    ),
  );
}

/// Loads SQLCipher instead of the system SQLite on Android.
void useSqlCipher() {
  open.overrideFor(OperatingSystem.android, openCipherOnAndroid);
}
```

Figure 4.4: The phone's sync engine

1-presentation-layer/mobile-app/lib/core/sync/sync_engine.dart — lines 85–130

```dart
Future<SyncOutcome> _run() async {
  _emit(SyncPhase.syncing);
  if (!_recovered) {
    await store.recoverInterrupted();
    _recovered = true;
  }
  List<OutboxData> inFlight = const [];
  try {
    final deviceId = await store.deviceId();
    for (var i = 0; i < _maxBatchesPerRun; i++) {
      inFlight = await store.takeBatch(batchSize);
      if (inFlight.isEmpty) break;
      await _push(deviceId, inFlight);
      inFlight = const [];
    }
    await _pull();

    final finished = _now();
    _failures = 0;
    _nextAttemptAt = null;
    await store.setMeta(
      MetaKey.lastSyncAt,
      finished.toUtc().toIso8601String(),
    );
    _emit(SyncPhase.idle, lastSyncAt: finished);
    return SyncOutcome.completed;
  } catch (e) {
    if (inFlight.isNotEmpty) {
      await store.releaseBatch(inFlight.map((o) => o.seq));
    }
    _failures++;
    var wait = backoffFor(_failures);
    // A rate-limited server says how long to wait; never retry sooner.
    final serverWait = e is ApiException ? e.retryAfter : null;
    if (serverWait != null && serverWait > wait) wait = serverWait;
    _nextAttemptAt = _now().add(wait);
    final offline = e is ApiException && e.isNetwork;
    _emit(
      offline ? SyncPhase.offline : SyncPhase.failed,
      error: e is ApiException
          ? e.message
          : 'Sync failed. It will try again.',
    );
    return offline ? SyncOutcome.offline : SyncOutcome.failed;
  }
}
```

Figure 4.5: The server applies each synced change once (idempotency)

3-application-logic/backend/src/services/sync/sync.service.ts — lines 124–196

```ts
private async apply(
  op: SyncOperationDto,
  deviceId: string,
  failedPatients: ReadonlySet<string>,
  user: AuthenticatedUser,
  ctx: RequestContext,
): Promise<SyncOperationResult> {
  const stored = await this.prisma.syncOperation.findUnique({
    where: { idempotencyKey: op.idempotencyKey },
  });
  if (stored) return this.replay(stored, op, user, ctx);

  let outcome: Omit<SyncOperationResult, 'idempotencyKey' | 'replayed'>;
  try {
    if (op.patientId && failedPatients.has(op.patientId)) {
      // Not stored: once the patient is fixed and synced, this can apply.
      return {
        idempotencyKey: op.idempotencyKey,
        replayed: false,
        result: 'REJECTED',
        entityType: op.entityType,
        error: {
          code: 'DEPENDENCY_FAILED',
          message:
            'The patient for this record could not be saved on the server',
        },
      };
    }
    outcome = await this.execute(op, user, ctx);
  } catch (e) {
    if (!(e instanceof Rejected)) throw e;
    outcome = {
      result: 'REJECTED',
      entityType: op.entityType,
      error: e.error,
    };
  }

  try {
    await this.prisma.syncOperation.create({
      data: {
        idempotencyKey: op.idempotencyKey,
        userId: user.id,
        deviceId,
        entityType: op.entityType,
        entityId: outcome.entityId ?? null,
        operation: op.operation,
        baseVersion: op.baseVersion ?? null,
        resultingVersion: outcome.version ?? null,
        result: outcome.result,
        conflictDetail: outcome.error
          ? (JSON.parse(
              JSON.stringify(outcome.error),
            ) as Prisma.InputJsonObject)
          : Prisma.JsonNull,
        clientTimestamp: new Date(op.clientTimestamp),
      },
    });
  } catch (e) {
    // Two concurrent batches with the same key: the first one wins.
    if (
      e instanceof Prisma.PrismaClientKnownRequestError &&
      e.code === 'P2002'
    ) {
      const winner = await this.prisma.syncOperation.findUniqueOrThrow({
        where: { idempotencyKey: op.idempotencyKey },
      });
      return this.replay(winner, op, user, ctx);
    }
    throw e;
  }
  return { idempotencyKey: op.idempotencyKey, replayed: false, ...outcome };
}
```

Figure 4.6: AES-256-GCM encryption of identifiers, and HMAC look-ups

3-application-logic/backend/src/persistence/crypto/field-crypto.ts lines 21–82

```ts
export class FieldCrypto {
  constructor(
    private readonly encryptionKey: Buffer,
    private readonly hmacKey: Buffer,
  ) {
    if (encryptionKey.length !== KEY_LENGTH || hmacKey.length !== KEY_LENGTH) {
      throw new Error('Field encryption and HMAC keys must each be 32 bytes');
    }
  }

  static fromEnv(env: Record<string, string | undefined>): FieldCrypto {
    const enc = env.FIELD_ENCRYPTION_KEY_BASE64;
    const mac = env.FIELD_HMAC_KEY_BASE64;
    if (!enc) throw new Error('FIELD_ENCRYPTION_KEY_BASE64 is not set');
    if (!mac) throw new Error('FIELD_HMAC_KEY_BASE64 is not set');
    return new FieldCrypto(
      Buffer.from(enc, 'base64'),
      Buffer.from(mac, 'base64'),
    );
  }

  /** Returns a standalone Uint8Array, the type Prisma uses for `Bytes` columns. */
  encrypt(plaintext: string): Uint8Array<ArrayBuffer> {
    const iv = randomBytes(IV_LENGTH);
    const cipher = createCipheriv('aes-256-gcm', this.encryptionKey, iv);
    const body = Buffer.concat([
      cipher.update(plaintext, 'utf8'),
      cipher.final(),
    ]);
    return new Uint8Array(
      Buffer.concat([Buffer.of(FORMAT_VERSION), iv, cipher.getAuthTag(), body]),
    );
  }

  decrypt(value: Uint8Array): string {
    const sealed = Buffer.from(
      value.buffer,
      value.byteOffset,
      value.byteLength,
    );
    if (
      sealed.length < 1 + IV_LENGTH + TAG_LENGTH ||
      sealed[0] !== FORMAT_VERSION
    ) {
      throw new Error('Unsupported encrypted field version or truncated value');
    }
    const iv = sealed.subarray(1, 1 + IV_LENGTH);
    const tag = sealed.subarray(1 + IV_LENGTH, 1 + IV_LENGTH + TAG_LENGTH);
    const body = sealed.subarray(1 + IV_LENGTH + TAG_LENGTH);
    const decipher = createDecipheriv('aes-256-gcm', this.encryptionKey, iv);
    decipher.setAuthTag(tag);
    return Buffer.concat([decipher.update(body), decipher.final()]).toString(
      'utf8',
    );
  }

  hmac(value: string): string {
    return createHmac('sha256', this.hmacKey)
      .update(value, 'utf8')
      .digest('hex');
  }
}
```

Figure 4.7: Sign-in with lockout and session tokens

3-application-logic/backend/src/services/auth/auth.service.ts lines 189–253

```ts
async login(
  emailInput: string,
  password: string,
  ctx: RequestContext,
): Promise<LoginResult> {
  const email = normalizeEmail(emailInput);
  const user = await this.prisma.user.findUnique({
    where: { email },
    include: USER_WITH_ACCESS,
  });

  if (!user) {
    await this.passwords.verifyDummy(password);
    await this.recordLogin(null, 'FAILURE', 'unknown_account', ctx);
    throw invalidCredentials();
  }
  if (user.status === 'DISABLED') {
    await this.passwords.verifyDummy(password);
    await this.recordLogin(user, 'FAILURE', 'disabled', ctx);
    throw invalidCredentials();
  }
  if (user.lockedUntil && user.lockedUntil > new Date()) {
    await this.recordLogin(user, 'DENIED', 'locked', ctx);
    throw accountLocked();
  }

  if (!(await this.passwords.verify(user.passwordHash, password))) {
    await this.registerFailedAttempt(user, ctx);
    throw invalidCredentials();
  }

  const familyId = randomUUID();
  const refresh = this.tokens.newRefreshToken();
  await this.prisma.$transaction(async (tx) => {
    await tx.user.update({
      where: { id: user.id },
      data: {
        failedLoginCount: 0,
        lockedUntil: null,
        lastLoginAt: new Date(),
      },
    });
    await tx.refreshToken.create({
      data: {
        userId: user.id,
        familyId,
        tokenHash: refresh.hash,
        expiresAt: this.tokens.refreshExpiry(),
      },
    });
    await this.recordLogin(user, 'SUCCESS', undefined, ctx, tx);
  });

  const roles = rolesOf(user);
  return {
    ...(await this.sessionTokens(user.id, familyId, refresh.token)),
    user: {
      id: user.id,
      email: user.email,
      displayName: user.displayName,
      roles,
      mustChangePassword: user.mustChangePassword,
    },
  };
}
```

Figure 4.8: Role-based access control: the permissions guard

3-application-logic/backend/src/gateway/access/permissions.guard.ts lines 29–44

```ts
async canActivate(context: ExecutionContext): Promise<boolean> {
  const targets = [context.getHandler(), context.getClass()];
  if (this.reflector.getAllAndOverride<boolean>(IS_PUBLIC, targets))
    return true;

  const required =
    this.reflector.getAllAndMerge<PermissionCode[]>(
      REQUIRED_PERMISSIONS,
      targets,
    ) ?? [];
  if (required.length === 0) return true;

  const req = context.switchToHttp().getRequest<AuthenticatedRequest>();
  const user = req.user;
  const missing = required.filter((p) => !user?.permissions.includes(p));
  if (missing.length === 0) return true;
```

Figure 4.9: TLS 1.3 only at the reverse proxy

2-api-gateway/reverse-proxy/nginx.conf lines 33–66

```
server {
  listen 443 ssl;
  http2 on;
  server_name api.example.org;

  ssl_certificate     /etc/nginx/certs/fullchain.pem;
  ssl_certificate_key /etc/nginx/certs/privkey.pem;
  ssl_protocols TLSv1.3;
  ssl_prefer_server_ciphers off;
  ssl_session_cache shared:SSL:10m;
  ssl_session_tickets off;

  add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
  add_header X-Content-Type-Options nosniff always;

  # Imaging and slide uploads stream straight through to the backend.
  client_max_body_size 2100m;
  proxy_request_buffering off;
  proxy_read_timeout 120s;

  location /api/ {
    proxy_pass http://pca_api;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
  }

  # Everything else is refused: the API is the only door.
  location / {
    return 404;
  }
}
```

Figure 4.10: Safe Harbor de-identification rules for the FHIR export

3-application-logic/backend/src/services/fhir/deidentify.ts lines 17–46

```ts
export const SAFE_HARBOR: readonly SafeHarborRule[] = [
  {
    n: 1,
    identifier: 'Names',
    inThisSystem: "Patient's given and family name (stored encrypted)",
    inTheExport: 'Removed. Names are never decrypted for an export.',
  },
  {
    n: 2,
    identifier: 'Geographic subdivisions smaller than a state',
    inThisSystem:
      "Patient's district; the facility's name, code, district and province",
    inTheExport:
      'Removed. Only the country (ZM) and the urban / peri-urban / rural class are kept.',
  },
  {
    n: 3,
    identifier:
      'All elements of dates (except year) directly related to a person; ages over 89',
    inThisSystem:
      'Date of birth, screening dates, biopsy dates, review, consent and AI dates',
    inTheExport:
      'Year only. From age 90 the birth year is removed too and the age is given as "90 or older".',
  },
  {
    n: 4,
    identifier: 'Telephone numbers',
    inThisSystem: "Patient's phone number (stored encrypted)",
    inTheExport: 'Removed.',
  },
```

Figure 4.11: Push notifications: the outbox sends each new notification once (ADR-014)

3-application-logic/backend/src/services/notifications/push/push-outbox.service.ts lines 84–145

```ts
  private async run(): Promise<PushRun> {
    const result: PushRun = {
      notifications: 0,
      sent: 0,
      failed: 0,
      removedDevices: 0,
    };
    if (!this.sender.enabled) return result;
    const claimed = await this.prisma.$queryRaw<Claimed[]>`
      UPDATE notifications SET pushed_at = now()
      WHERE id IN (
        SELECT id FROM notifications
        WHERE pushed_at IS NULL
          AND created_at > now() - make_interval(mins => ${this.config.push.maxAgeMinutes}::int)
        ORDER BY created_at
        LIMIT ${BATCH}
        FOR UPDATE SKIP LOCKED)
      RETURNING id, user_id AS "userId", type, title, body`;
    result.notifications = claimed.length;
    if (claimed.length === 0) return result;

    // Disabled accounts get nothing; a locked one (wrong passwords) still does.
    const devices = await this.prisma.pushDevice.findMany({
      where: {
        userId: { in: [...new Set(claimed.map((n) => n.userId))] },
        user: { status: { not: 'DISABLED' } },
      },
      select: { id: true, userId: true, token: true },
    });
    const invalid = new Set<string>();
    for (const notification of claimed) {
      for (const device of devices) {
        if (device.userId !== notification.userId || invalid.has(device.id)) {
          continue;
        }
        const outcome = await this.sender.send({
          token: device.token,
          title: notification.title,
          body: notification.body,
          data: { notificationId: notification.id, type: notification.type },
        });
        if (outcome === 'sent') {
          result.sent++;
        } else {
          result.failed++;
          if (outcome === 'invalid-token') invalid.add(device.id);
        }
      }
    }
    if (invalid.size > 0) {
      const { count } = await this.prisma.pushDevice.deleteMany({
        where: { id: { in: [...invalid] } },
      });
      result.removedDevices = count;
    }
    if (result.failed > 0) {
      this.logger.warn(
        `Push: ${result.sent} sent, ${result.failed} not sent; ${result.removedDevices} phones no longer known to Firebase were removed`,
      );
    }
    return result;
  }
```

Objective 2: the multi-modal AI inference engine (the pipeline is built; the trained models are not, so the modules are labelled mocks)

Figure 4.12: Requesting an analysis: consent first

3-application-logic/backend/src/services/ai/ai.service.ts lines 218–256

```
218  async request(
219    patientId: string,
220    user: AuthenticatedUser,
221    ctx: RequestContext,
222  ): Promise<AiJobView> {
223    if (!this.broker.enabled) {
224      throw new ServiceUnavailableException({
225        code: 'AI_UNAVAILABLE',
226        message: 'AI analysis is switched off on this server',
227      });
228    }
229    const patient = await this.patients.requireInFacility(user, patientId);
230    if (!(await this.clinical.hasActiveConsent(patient.id, 'AI_ANALYSIS'))) {
231      throw new ConflictException({
232        code: 'CONSENT_REQUIRED',
233        message:
234          'The patient has not consented to AI analysis. Record their consent first.',
235      });
236    }
237    const inProgress = await this.prisma.aiJob.findFirst({
238      where: { patientId: patient.id, status: { in: ['QUEUED', 'RUNNING'] } },
239    });
240    if (inProgress) {
241      throw new ConflictException({
242        code: 'AI_JOB_IN_PROGRESS',
243        message: 'An analysis for this patient is already running',
244        details: { jobId: inProgress.id },
245      });
246    }
247    const record = await this.prisma.clinicalRecord.findFirst({
248      where: { patientId: patient.id },
249      orderBy: [{ encounterDate: 'desc' }, { createdAt: 'desc' }],
250    });
251    if (!record) {
252      throw new ConflictException({
253        code: 'CLINICAL_RECORD_REQUIRED',
254        message: 'Add a screening record before requesting an analysis',
255      });
256    }
```

Figure 4.13: The model router and the fusion step

4-ai-intelligence-layer/ai-services/app/router.py — lines 43–76

```python
    def infer(self, request: InferenceRequest) -> InferenceResult:
        used: list[ModelProvider] = []
        skipped: list[SkippedModule] = []
        outputs: dict[str, ModuleOutput] = {}

        for module in self._modules:
            reason = module.unavailable_reason(request.inputs)
            if reason:
                skipped.append(SkippedModule(module=module.module, reason=reason))
                continue
            outputs[module.module] = module.run(request, dict(outputs))
            used.append(module)

        if not used:
            raise InsufficientInputsError(skipped)

        probability: float | None = None
        interval: tuple[float, float] | None = None
        if any(o.probability is not None for o in outputs.values()):
            fused = self._fusion.run(request, dict(outputs))
            used.append(self._fusion)
            probability = fused.probability
            if probability is not None:
                interval = (
                    max(0.0, round(probability - 0.1, 2)),
                    min(1.0, round(probability + 0.1, 2)),
                )
        else:
            skipped.append(
                SkippedModule(
                    module=self._fusion.module,
                    reason="No module produced a probability to combine",
                )
            )
```

Figure 4.14: Fairness check across groups (AUC gap above 0.05 flagged)

3-application-logic/backend/src/services/ai/fairness.ts lines 68–107

```ts
export function fairnessFrom(evaluation: unknown): FairnessReport | null {
  if (!isRecord(evaluation) || !isRecord(evaluation.byGroup)) return null;
  const dimensions: FairnessDimension[] = [];
  for (const [name, raw] of Object.entries(evaluation.byGroup)) {
    if (!isRecord(raw)) continue;
    const groups = Object.entries(raw).map(([g, figures]) => group(g, figures));
    const aucs = groups.filter((g) => g.compared).map((g) => g.auc as number);
    if (aucs.length < 2) {
      dimensions.push({
        name,
        groups,
        aucGap: null,
        flagged: false,
        note: 'Fewer than two groups have enough test cases, so they cannot be compared.',
      });
      continue;
    }
    // Rounded to the 3 decimals shown, so 0.90 - 0.85 is exactly 0.05 (not
    // 0.05000000000000004) and is not flagged.
    const gap =
      Math.round((Math.max(...aucs) - Math.min(...aucs)) * 1000) / 1000;
    const flagged = gap > FAIRNESS_AUC_GAP;
    dimensions.push({
      name,
      groups,
      aucGap: gap,
      flagged,
      note: flagged
        ? `The AUC gap is above ${FAIRNESS_AUC_GAP}: remediation is needed before relying on this model for all groups.`
        : null,
    });
  }
  if (dimensions.length === 0) return null;
  return {
    threshold: FAIRNESS_AUC_GAP,
    minCases: FAIRNESS_MIN_CASES,
    dimensions,
    flagged: dimensions.some((d) => d.flagged),
  };
}
```

Objective 3: the patient education chatbot

Figure 4.15: Safety rules, checked before every answer

3-application-logic/backend/src/services/chatbot/chat-safety.ts lines 52–87

```
export function checkQuestion(
  text: string,
  audience: ChatAudience,
): SafetyDecision {
  if (SELF_HARM.test(text)) {
    return {
      safety: 'URGENT_CARE',
      reason: 'self-harm',
      text: FIXED_TEXT.urgentSelfHarm,
    };
  }
  if (EMERGENCY.test(text)) {
    return {
      safety: 'URGENT_CARE',
      reason: 'emergency',
      text: FIXED_TEXT.urgentMedical,
    };
  }
  if (MEDICINES.test(text)) {
    return {
      safety: 'DECLINED',
      reason: 'medicines',
      text: FIXED_TEXT.medicines,
    };
  }
  // Clinicians may ask what a value means in general; patients are never
  // told what their own results mean (their clinician explains them).
  if (audience === 'patient' && OWN_RESULTS.test(text)) {
    return {
      safety: 'DECLINED',
      reason: 'own-results',
      text: FIXED_TEXT.ownResults,
    };
  }
  return { safety: 'OK', reason: 'ok' };
}
```

Figure 4.16: The chatbot service: safety, small talk, then the knowledge base

3-application-logic/backend/src/services/chatbot/chatbot.service.ts — lines 270–327

```ts
  /** Safety check, then the quoted answer, then the output check. */
  private async answer(
    text: string,
    audience: ChatAudience,
    language: string,
    earlier: StoredMessage[],
  ): Promise<{ answer: StoredMessage; reason: string }> {
    const fixed = (safety: ChatSafety, fixedText: string, reason: string) => ({
      answer: {
        id: randomUUID(),
        role: 'assistant' as const,
        text: fixedText,
        at: new Date(),
        sources: [],
        safety,
        mode: 'FIXED' as const,
      },
      reason,
    });
    const decision = checkQuestion(text, audience);
    if (decision.safety !== 'OK') {
      return fixed(decision.safety, decision.text!, decision.reason);
    }
    // The number of earlier questions varies repeated replies ("Hi again!",
    // another joke).
    const friendly = smallTalk(
      text,
      audience,
      earlier.filter((m) => m.role === 'user').length,
    );
    if (friendly) return fixed('OK', friendly, 'small-talk');
    const noSource =
      audience === 'patient'
        ? FIXED_TEXT.noSourcePatient
        : FIXED_TEXT.noSourceClinician;
    let result;
    try {
      result = await this.ai.chatAnswer({
        question: text,
        audience,
        language,
        // The last three questions and answers, for follow-ups.
        history: earlier.slice(-6).map((m) => ({ role: m.role, text: m.text })),
      });
    } catch (err) {
      if (err instanceof AiServiceError && err.status === 409) {
        throw new ConflictException({
          code: 'LANGUAGE_NOT_AVAILABLE',
          message: 'There is no verified content in this language yet.',
        });
      }
      throw new ServiceUnavailableException({
        code: 'CHAT_UNAVAILABLE',
        message: 'The assistant is not available right now. Please try later.',
      });
    }
    if (!result.matched) return fixed('NO_SOURCE', noSource, 'no-match');
    const check = checkAnswer(result.text, result.sources.length);
```

Figure 4.17: Retrieval, step 1: BM25 keyword search of the reviewed knowledge base

4-ai-intelligence-layer/ai-services/app/chat/retrieve.py — lines 65–100

```python
class Retriever:
    """BM25 over the passages one audience may see."""

    K1 = 1.5
    B = 0.75

    def __init__(self, kb: KnowledgeBase, audience: Audience, language: str) -> None:
        # Clinicians may also be given the plain patient information.
        allowed = {"patient"} if audience == "patient" else {"patient", "clinician"}
        self.passages = [p for p in kb.passages if p.audience in allowed and p.language == language]
        # The title and heading count twice: they say what the passage is about.
        self._docs = [
            terms(f"{p.title} {p.heading} {p.title} {p.heading} {p.body}") for p in self.passages
        ]
        self._freq = [Counter(d) for d in self._docs]
        n = len(self._docs)
        self._avg_len = sum(len(d) for d in self._docs) / n if n else 0.0
        df: Counter[str] = Counter()
        for d in self._docs:
            df.update(set(d))
        self.idf = {t: math.log(1 + (n - c + 0.5) / (c + 0.5)) for t, c in df.items()}

    def search(self, question: str, top_k: int = 5) -> list[Hit]:
        query = query_terms(question)
        hits: list[Hit] = []
        for passage, doc, freq in zip(self.passages, self._docs, self._freq, strict=True):
            score = 0.0
            for t in query:
                f = freq.get(t, 0)
                if f:
                    norm = self.K1 * (1 - self.B + self.B * len(doc) / self._avg_len)
                    score += self.idf[t] * f * (self.K1 + 1) / (f + norm)
            if score > 0:
                hits.append(Hit(passage, score))
        hits.sort(key=lambda h: h.score, reverse=True)
        return hits[:top_k]
```

Figure 4.18: Retrieval, step 2: the keywords decide, MedCPT chooses the article (ADR-013)

4-ai-intelligence-layer/ai-services/app/chat/answer.py — lines 132–194

```python
def _rank(
    retriever: Retriever, question: str, text: str, meaning: MeaningScorer | None
) -> tuple[list[Hit], list[Hit]] | None:
    """The passages in order of relevance, and the ones to quote; None when the
    knowledge base does not cover the question. `text` is the question, or a
    short follow-up read together with the previous question."""
    if meaning is None:
        hits = retriever.search(text)
        if not hits or hits[0].score < MIN_SCORE:
            return None
        # Whole passages are quoted: each knowledge base section is written to
        # stand on its own, so a quote never loses the sentence it depends on.
        quoted = [hits[0]]
        if (
            len(hits) > 1
            and hits[1].score >= SECOND_PASSAGE_RATIO * hits[0].score
            and len(hits[0].passage.body) + len(hits[1].passage.body) <= MAX_CHARS
        ):
            quoted.append(hits[1])
        return hits, quoted

    # Meaning search: the keywords decide whether the question is covered.
    # MedCPT tells articles apart well, but scores the sections of one article
    # within a point or two of each other, where the keywords choose better
    # ("What does PI-RADS 4 mean?" is answered by the five categories). So the
    # articles come in order of their closest passage, and inside an article
    # the passages keep their keyword order (candidates are in BM25 order, and
    # the sort is stable).
    hits = retriever.search(text, top_k=len(retriever.passages))
    candidates = [h for h in hits if h.score >= MIN_SCORE]
    if not candidates:
        return None
    closeness = meaning.scores(text, [h.passage for h in candidates])
    article_best: dict[str, float] = {}
    for hit, close in zip(candidates, closeness, strict=True):
        article = hit.passage.article_id
        article_best[article] = max(article_best.get(article, close), close)
    ordered = sorted(
        zip(candidates, closeness, strict=True),
        key=lambda pair: -article_best[pair[0].passage.article_id],
    )
    best = ordered[0][0]
    best_closeness = article_best[best.passage.article_id]
    if best_closeness < MEANING_FLOOR:
        return None
    # A short question read with the previous one must itself be close to the
    # article: "Does it hurt?" after the DRE is, "What is the treatment for
    # malaria?" after the DRE is not (it is a new topic, and not covered).
    if text != question:
        same_article = [
            h.passage for h, _ in ordered if h.passage.article_id == best.passage.article_id
        ]
        if max(meaning.scores(question, same_article)) < MEANING_FLOOR:
            return None
    quoted = [best]
    if (
        len(ordered) > 1
        and ordered[1][1] >= best_closeness - MEANING_SECOND_GAP
        and len(best.passage.body) + len(ordered[1][0].passage.body) <= MAX_CHARS
    ):
        quoted.append(ordered[1][0])
    ranked = [h for h, _ in ordered] + [h for h in hits if h.score < MIN_SCORE]
    return ranked, quoted
```

Figure 4.19: The MedCPT encoder (PubMedBERT, 12 layers) written with numpy

4-ai-intelligence-layer/ai-services/app/chat/embeddings.py — lines 171–203

```python
    def _encode(self, ids: Any, mask: Any, types: Any) -> Any:
        np = self._np
        batch, length = ids.shape
        h = (
            self.w["embeddings.word_embeddings.weight"][ids]
            + self.w["embeddings.position_embeddings.weight"][:length]
            + self.w["embeddings.token_type_embeddings.weight"][types]
        )
        h = self._norm(h, "embeddings.LayerNorm")
        size = h.shape[-1] // self.heads
        # Padding gets no attention.
        blocked = ((1.0 - mask[:, None, None, :]) * np.finfo(np.float32).min).astype(np.float32)

        def heads(x: Any) -> Any:
            return x.reshape(batch, length, self.heads, size).transpose(0, 2, 1, 3)

        for i in range(self.layers):
            p = f"encoder.layer.{i}"
            q = heads(self._linear(h, f"{p}.attention.self.query"))
            k = heads(self._linear(h, f"{p}.attention.self.key"))
            v = heads(self._linear(h, f"{p}.attention.self.value"))
            scores = q @ k.transpose(0, 1, 3, 2) / math.sqrt(size) + blocked
            scores = np.exp(scores - scores.max(-1, keepdims=True))
            probs = scores / scores.sum(-1, keepdims=True)
            context = (probs @ v).transpose(0, 2, 1, 3).reshape(batch, length, -1)
            h = self._norm(
                self._linear(context, f"{p}.attention.output.dense") + h,
                f"{p}.attention.output.LayerNorm",
            )
            inner = self._linear(h, f"{p}.intermediate.dense")
            inner = 0.5 * inner * (1.0 + _erf(inner / math.sqrt(2.0)))
            h = self._norm(self._linear(inner, f"{p}.output.dense") + h, f"{p}.output.LayerNorm")
        return h
```

Figure 4.20: Writing the answer from the passages

4-ai-intelligence-layer/ai-services/app/chat/answer.py — lines 197–251

```python
def answer(
    kb: KnowledgeBase,
    request: ChatAnswerRequest,
    writer: ClaudeWriter | None = None,
    meaning: MeaningScorer | None = None,
) -> ChatAnswerResult:
    if request.language not in kb.languages():
        raise LanguageNotAvailableError(request.language)
    audience: Audience = request.audience
    retriever = Retriever(kb, audience, request.language)
    info = KnowledgeBaseInfo(version=kb.version, reviewStatus=kb.review_status)
    unmatched = ChatAnswerResult(
        matched=False, text=None, passages=[], sources=[], knowledgeBase=info
    )
    text = _search_text(request)
    try:
        ranking = _rank(retriever, request.question, text, meaning)
    except MeaningBusyError:
        # Every slot of the meaning search is taken (heavy load): keywords
        # only, as before ADR-013, so the answer still comes within 2 s.
        ranking = _rank(retriever, request.question, text, None)
    if ranking is None:
        return unmatched
    hits, quoted = ranking

    if writer is not None:
        given = hits[:PASSAGES_FOR_CLAUDE]
        written = writer.write(
            request.question,
            audience,
            [h.passage for h in given],
            [Turn(role=t.role, text=t.text) for t in request.history],
        )
        if written is not None and not written.covered:
            return unmatched.model_copy(update={"mode": "GENERATED", "model": written.model})
        if written is not None and written.cited:
            used = [given[i] for i in written.cited]
            return ChatAnswerResult(
                mode="GENERATED",
                matched=True,
                text=written.text,
                passages=[_passage_out(h) for h in used],
                sources=_sources([h.passage for h in used]),
                knowledgeBase=_info(kb, [h.passage for h in used]),
                model=written.model,
            )
        # No usable answer from Claude: quote the passages instead.

    return ChatAnswerResult(
        matched=True,
        text=" ".join(h.passage.body for h in quoted),
        passages=[_passage_out(h) for h in quoted],
        sources=_sources([h.passage for h in quoted]),
        knowledgeBase=_info(kb, [h.passage for h in quoted]),
    )
```

Figure 4.21: The app's chat controller

1-presentation-layer/mobile-app/lib/features/chat/application/chat_controller.dart lines 82–117

```dart
class ChatController extends Notifier<ChatState> {
  @override
  ChatState build() => const ChatState();

  Future<void> ask(String text) async {
    final question = text.trim();
    if (question.isEmpty || state.sending || state.loading) return;
    state = state.copyWith(sending: true, clearError: true);
    final api = ref.read(chatApiProvider);
    try {
      final id = state.conversationId ?? await api.start();
      final (asked, answer) = await api.ask(id, question);
      state = state.copyWith(
        conversationId: id,
        messages: [...state.messages, asked, answer],
        sending: false,
      );
      ref.invalidate(chatHistoryProvider);
    } catch (e) {
      state = state.copyWith(sending: false, error: chatErrorText(e));
    }
  }

  /// Opens an earlier conversation so it can be read and continued.
  Future<void> open(String conversationId) async {
    if (state.sending || conversationId == state.conversationId) return;
    state = ChatState(conversationId: conversationId, loading: true);
    try {
      final messages = await ref.read(chatApiProvider).messages(conversationId);
      if (state.conversationId != conversationId) return;
      state = ChatState(conversationId: conversationId, messages: messages);
    } catch (e) {
      if (state.conversationId != conversationId) return;
      state = ChatState(error: chatErrorText(e));
    }
  }
```

Figure 4.22: Voice input with the phone's speech service

1-presentation-layer/mobile-app/lib/features/chat/application/voice_input.dart lines 74–102

```dart
Future<void> listen({
  required void Function(String words) onWords,
  required void Function(double level) onLevel,
  required void Function() onDone,
}) async {
  _onDone = onDone;
  // The answers are in English: use the phone's English locale, or
  // British English when the phone is set to another language.
  final system = await _speech.systemLocale();
  final locale = system != null && system.localeId.startsWith('en')
      ? system.localeId
      : 'en_GB';
  await _speech.listen(
    onResult: (result) {
      onWords(result.recognizedWords);
      if (result.finalResult) _finish();
    },
    // Android reports roughly -2 to 10 dB.
    onSoundLevelChange: (level) => onLevel(((level + 2) / 12).clamp(0, 1)),
    listenOptions: stt.SpeechListenOptions(
      listenFor: const Duration(seconds: 60),
      pauseFor: const Duration(seconds: 3),
      localeId: locale,
      partialResults: true,
      cancelOnError: true,
      listenMode: stt.ListenMode.dictation,
    ),
  );
}
```

Figure 4.23: Reading aloud for people who cannot read

1-presentation-layer/mobile-app/lib/shared/audio/read_aloud.dart — lines 110–139

```dart
class ReadAloudController extends Notifier<ReadAloudState> {
  int _run = 0;
  List<String> _parts = const [];

  ReadAloud get _voice => ref.read(readAloudProvider);

  @override
  ReadAloudState build() => const ReadAloudState();

  Future<void> play(String id, List<String> parts, {int from = 0}) async {
    final run = ++_run;
    await _voice.stop();
    _parts = parts;
    if (!ref.mounted) return;
    await _voice.setRate(state.slow ? slowReadingRate : normalReadingRate);
    for (var i = from; i < parts.length; i++) {
      // Stopped, or another text started, or the app is closing.
      if (!ref.mounted || run != _run) return;
      state = ReadAloudState(
        id: id,
        part: i,
        total: parts.length,
        slow: state.slow,
      );
      await _voice.speak(parts[i]);
    }
    if (ref.mounted && run == _run) state = ReadAloudState(slow: state.slow);
  }

  Future<void> pause() async {
```

Objective 5: ethics, privacy and compliance

Figure 4.24: The audit log's hash chain (database trigger)

5-data-persistence/postgresql/migrations/20260923203337_constraints_and_audit/migration.sql lines 75–92

```
75 CREATE OR REPLACE FUNCTION audit_logs_before_insert()
76 RETURNS trigger LANGUAGE plpgsql AS $$
77 DECLARE
78   v_prev text;
79 BEGIN
80   PERFORM pg_advisory_xact_lock(hashtext('audit_logs_chain'));
81   NEW.seq := nextval(pg_get_serial_sequence('audit_logs', 'seq'));
82   NEW.occurred_at := date_trunc('milliseconds', coalesce(NEW.occurred_at, now()));
83   SELECT row_hash INTO v_prev FROM audit_logs ORDER BY seq DESC LIMIT 1;
84   NEW.prev_hash := coalesce(v_prev, repeat('0', 64));
85   NEW.row_hash := audit_logs_row_hash(NEW.prev_hash, NEW);
86   RETURN NEW;
87 END;
88 $$;
89
90 CREATE TRIGGER audit_logs_chain
91   BEFORE INSERT ON audit_logs
92   FOR EACH ROW EXECUTE FUNCTION audit_logs_before_insert();
```

Figures 4.6 (encryption), 4.10 (de-identification) and 4.14 (fairness) also belong to this objective. **Objective 4** (data collection and model training) has no code yet: no training data or trained model exists in the prototype.

More code pictures in report/img, for example the small-talk replies (`code-chat-small-talk.png`), the knowledge base (`code-ai-knowledge-base.png`), the assistant's face drawn in code (`code-app-bot-painter.png`), the user model (`code-backend-prisma-user.png`) and the admin delete dialog (`code-admin-delete-dialog.png`).